Bachelor thesis

Artificial Intelligence

Radboud University

Tackling non-stationarity in ERP classification: updating strategies for an adaptive Block-Toeplitz LDA classifier

Author:
Soz Raheem
s1096881

First supervisor:
dr. M.W. Tangermann
Donders Centre for Cognition,
Donders Institue
michael.tangermann@donders.ru.nl

Second reader:
dr. W.A.J.J. Wiegerinck
Donders Centre for Cognition,
Donders Institue
wim.wiegerinck@donders.ru.nl

June 26, 2025

Abstract

Brain activity naturally changes over time, posing a challenge for classification in brain computer interfaces (BCI). The recently invented Block-Toeplitz linear discriminant analysis (BT-LDA) has shown promising improvements over the current state-of-the-art classifiers on event-related potential (ERP) data. However, its performance over time is threatened by non-stationary brain signals, necessitating updating strategies. These strategies incrementally update the classifier, striving to achieve more accurate and up-to-date performance. I have explored several adaptation strategies with the aim to identify an optimal strategy that enhances and preserves the long-term accuracy of the adaptive BT-LDA. *Methods:* One static and two different adaptive classifiers have been constructed and compared against two additional baselines. The classifiers were validated in an online simulation, using data of 10 patients from a language rehabilitation experiment. *Results:* The Adaptive Window BT-LDA consistently outperformed both baselines in epoch-wise accuracy, showing statistically significant improvements. Trial-wise accuracy also improved compared to one of the baselines, though results were more variable. Additionally, the optimal update coefficients (UCs) varied across subjects, suggesting that subject-specific or dynamic updating could further enhance performance. *Conclusion:* These findings indicate that update strategies significantly influence long-term classifier performance. While results depend on the dataset, which in this thesis is derived from an aphasia rehabilitation protocol, they point toward the potential of adaptive BT-LDA methods in improving classification performance in ERP data. This could have important implications for real-time applications in BCIs and neurorehabilitation.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Michael, for continuously providing guidance and constructive feedback throughout the process. I have learned a lot in such a short period and I truly enjoyed working on this project. Thank you for making this journey such a valuable experience.

I am also thankful to Jordy for his technical assistance and helpful insights, and to Matthias for guiding me through the software of the aphasia experiment.

Furthermore, I would like to thank the fellow students in my thesis group, Sophia, Lucy, and Joep, for contributing ideas and thinking along throughout the process. Lucy in particular provided great help in setting up the codebase for my project.

I am appreciative of my friends and fellow students for their support through successes and challenges. In particular, I would like to thank Nadira, Alizeh and Cristian for their thoughtful advice and encouragement.

I would also like to thank my family. Especially my mother has helped me greatly through her love and support, as she always has, and I can never thank her enough for that.

Above all, I am deeply grateful to God. The opportunity to study and grow through higher education is a gift I do not take for granted. I hope to carry this gift forward by using what I have learned to help those in need.

Contents

1 Introduction **4**
1.1 Brain-Computer Interfaces 4
1.2 Classification and Regularization Techniques 4
1.3 The Non-Stationarity Problem 5
1.4 Adaptation Strategies 5
1.5 Aim of the Project 6

2 Background **8**
2.1 Brain Signals and Event-Related Potentials 8
2.2 The BCI Pipeline 9
2.3 Linear Discriminant Analysis 10
2.3.1 Shrinkage Regularization 11
2.4 Block-Toeplitz LDA 11
2.4.1 Regularization 12
2.5 BT-LDA in the Pipeline 12
2.5.1 Adaptation in BT-LDA 12

3 Methods **13**
3.1 Dataset 13
3.1.1 Participants 13
3.1.2 Experimental protocol 13
3.1.3 Offline and online sessions 14
3.1.4 Classifier in the protocol 15
3.1.5 Conditions 15
3.1.6 Dynamic stopping 15
3.2 Pipeline Design 15
3.2.1 Data preprocessing 16
3.2.2 Feature extraction 16
3.2.3 Classification 17
3.3 Related Work on Adaptive Classifiers 18
3.4 Proposed Adaptation Strategies 19
3.4.1 Adaptive Window BT-LDA 19
3.4.2 Adaptive CC BT-LDA 20
3.4.3 Optimizing the UC-values for Adaptive CC BT-LDA 21
3.4.4 Additional variability in the online simulations 22
3.5 Evaluation 22
3.5.1 Calibration sessions 22
3.5.2 Statistical Analysis 24

4 Results **25**
4.1 Hyperparameters . . . 25
4.1.1 Time Intervals . . . 25
4.1.2 Moving Window Size for Adaptive Window BT-LDA . . . 25
4.2 Grid search on UC-pairs for Adaptive CC BT-LDA . . . 25
4.2.1 Representative patient to reduce UC-ranges . . . 25
4.2.2 Optimizing the general UC-pair . . . 25
4.2.3 Optimizing individual UC-pairs . . . 28
4.3 Comparing the Adaptive Classifiers . . . 29
4.3.1 Window Size for the Adaptive Window BT-LDA . . . 29
4.3.2 Grand Average Epoch-Wise Performance Over Sessions . . . 30
4.3.3 Statistical Analysis . . . 34

5 Discussion **35**
5.1 Optimizing the Update Coefficient . . . 35
5.1.1 The UC-Grid of the Representative Patient . . . 35
5.1.2 The UC-Grid of the Grouped Patients . . . 36
5.2 Optimal Updating Strategy . . . 36
5.2.1 Evaluation Methods . . . 37
5.3 Limitations and Future Directions . . . 37
5.3.1 Second Baseline: Adaptive CC sLDA . . . 37
5.3.2 Offline Analysis . . . 37
5.3.3 Intra-session variability in the dataset . . . 38
5.4 Future Work . . . 38
5.4.1 Improving the classification in aphasia rehabilitation . . . 38
5.5 Conclusion . . . 38

6 Source Code **40**

A Dataset Information **44**
A.1 Experimental Conditions per Patient . . . 44

B Hyperparameters **45**
B.1 Optimized Time Intervals for BT-LDA . . . 45

C Representative Patient **46**

D Additional Results **47**
D.1 UC-Pair Grid Search for Every Patient . . . 47

Chapter 1

Introduction

By connecting the human brain with machines, activities can take place that previously seemed impossible. A brain-computer interface (BCI) allows a person to communicate or control a device, using only their mind [1]. These activities can be especially impactful for disabled patients with impairments in their nervous system due to stroke, trauma, or neuromuscular disorders, such as Parkinson's disease and amyotrophic lateral sclerosis [2, 3]. BCIs enable these patients to restore their lost functions through various applications, including BCI-based prosthetics [4, 5], robotic arms [5], spelling devices [6], and remotely controlled wheelchairs [6], improving their quality of life.

Not only have BCIs been deployed to enable direct communication and control, but neurorehabilitation methods have also been developed [6]. Successful applications in the past include motor rehabilitation [4, 5] and rehabilitation of language disorders, as in the work by Musso et al. [7], which demonstrated significant medium to large improvements in language among aphasic patients.

1.1 Brain-Computer Interfaces

More specifically, a BCI is a system that records a participant's brain activity, processes that information, and produces feedback about the user's mental state. Several models exist that describe the main components of a BCI system. For this study, I follow the framework described in [8], which divides a BCI system into five components: (i) signal acquisition, (ii) preprocessing, (iii) feature extraction, (iv) classification and (v) device control.

Brain activity can be measured through an electroencephalogram (EEG), where different types of signals are analyzed. One of them is event-related potentials (ERPs), which are voltage potentials time-locked to a stimulus onset. These ERPs are measured in numerous BCI applications, where the participant is presented with several stimuli, of which a selection consists of target stimuli among other non-target stimuli. The expectation that the target stimuli produce different ERPs than non-target ones serves as the criterion for the classifier in the decoding model. The aim of the classification is then to predict whether a recorded event is a target or non-target stimulus.

1.2 Classification and Regularization Techniques

Currently, linear discriminant analysis (LDA) is one of the most widely used classifiers for decoding EEG data [9]. This method projects data onto a one-dimensional subspace,

where the projected classes are separated maximally while the within-class covariance is minimized (see Chapter 2 for more details). This projection requires an accurate estimation of the class means and the class covariance matrices, both of which are based on the training data. However, in most BCI applications, a large training dataset is difficult to obtain, as the dataset is only sampled per participant due to individual variability in brain activity data [10]. This results in less reliable estimates of the covariance matrices, causing the LDA to perform sub-optimally [11].

A solution to this problem is the implementation of regularization techniques, such as shrinkage regularization, where the estimated covariance matrix is regularized by shrinkage. This shrinkage LDA (sLDA) showed an increased performance over standard LDA on small datasets [11]. A more recent approach, introduced by Sosulski and Tangermann [12], involves enforcing a Block-Toeplitz structure within the covariance matrix. Compared to standard LDA and sLDA, Block-Toeplitz-LDA (BT-LDA) showed significant improvements in performance on ERP data [12]. Therefore, in this study, BT-LDA will be used as a basis to compare different updating strategies.

1.3 The Non-Stationarity Problem

Among the small datasets available for training the classifier, another challenge arises from the non-stationarity in brain activity. Over time, ERP recordings of participants change in structure due to both technical reasons, including environmental noise, and psychophysiological reasons, such as fatigue, motivational changes, and changing strategies when performing a specific task [13]. The latter is especially important and even encouraged in neurorehabilitation applications [5], for instance, in the aphasia rehabilitation paradigm by Musso and colleagues [7], where reinforcement of strategies of aphasic patients is assumed to be the underlying mechanism of the improved language recovery results.

As a consequence of non-stationarity, the trained classifier becomes outdated over time, resulting in a decreased accuracy [14]. In motor imagery-based BCIs, non-stationarity is a major factor behind the well-known issue of BCI illiteracy, where for a significant portion of users, BCI control is not possible [15]. This issue arises from the classifier's inability to generalize learned patterns from *offline* sessions (where the data is used to train the classifier) to *online* sessions (where the classifier is applied to new test data) due to changing data distributions [14]. This variability is seen not only in the transition from the training session to online sessions but also within online sessions [16].

1.4 Adaptation Strategies

The decrease in performance caused by non-stationarity necessitates adaptation strategies, which continuously update the BCI across sessions or even within sessions. Adaptation takes two forms: user adaptation and machine adaptation [13]. User adaptation is encouraged in rehabilitation paradigms, where, through adaptive feedback, patients learn to produce the desired brain activity that the classifier can decode [5, 7]. However, this form of adaptation is difficult to maintain with patients, as they typically show unstable brain activity [13]. The focus of this study will be on adapting the machine instead. Machine adaptation can be implemented in two different phases of the BCI model: (iii) feature

extraction and (iv) classification [17]. In this work, I focus on adaptation strategies for classification.

Adaptive classifiers can be updated in a supervised, unsupervised, or semi-supervised manner. The supervised approach can be realized when novel data obtained from online use are accompanied by their corresponding labels. The dataset used for this thesis contains such online labels, allowing supervised updating. Therefore, I will consider supervised adaptation strategies.

Currently, different updating strategies exist for adaptive classifiers. One technique is to update the classifier through a moving window, modifying the training set by continuously adding recent data and forgetting old data. A more advanced technique is to assign weights to the data, depending on their novelty. For instance, a classifier can continuously be updated through a convex combination of the current classifier with a new classifier, which is trained with novel obtained data. This approach was examined by Vidaurre and Blankertz [18], where the LDA classifier was adapted by adjustments to the covariance matrix and class means.

Adaptive classifiers have shown improved accuracy scores over static ones [9], which are only trained once and do not adapt to changing brain activity. In Chapter 3, existing strategies in the literature are discussed. However, the mentioned strategies were only applied to standard LDA or sLDA. With significantly higher accuracy scores than standard LDA and sLDA on ERP data [12], the recently invented BT-LDA becomes an interesting basis for the implementation of these strategies. The unexplored combination of existing updating strategies with BT-LDA on patient ERP data can potentially result in surpassing classification performance compared to the current state of the art.

1.5 Aim of the Project

This thesis aims to find an optimal adaptation strategy for BT-LDA through the following research question: How do different updating strategies influence the classification accuracy of adaptive Block-Toeplitz-LDA on non-stationary patient ERP data?

To answer the research question, I implemented a selection of adaptation strategies and constructed four versions of BT-LDA. Two of them are fixed, i.e., they do not adapt to changing brain activity, and the other two are adaptive. The proposed updating strategies include a moving window and a convex combination (Chapter 3). The latter involves a hyperparameter, the so-called update coefficient, which represents the influence of novel data. As part of the updating strategy, the second question of this thesis is therefore defined as follows: What is an optimal update coefficient for the adaptive Block-Toeplitz-LDA?

The classifier versions were deployed in an online simulation, using the data from the aphasia rehabilitation therapy by Musso et al. [7]. The classification performances are assessed trough the *epoch-wise* and *trial-wise* accuracy, both of which are explained in Chapter 3. The results of the different versions are compared with one of the fixed versions of BT-LDA, which serves as a baseline. A second baseline is the adaptive classifier used in the aphasia experiment [7]. Note that the performance relies on both the classifier (including the use of any adaptation strategy) and the dataset.

The remainder of this thesis is organized as follows. In Chapter 2, I will lay the theoretical foundation for this thesis. In Chapter 3, I provide details about the dataset, after which I discuss existing work on adaptive classifiers, propose different versions of BT-LDA and explain the evaluation protocol. The results of the offline analysis are presented in Chapter 4 and discussed in Chapter 5, which also highlights the limitations of this study and explores potential future research directions based on this work. Lastly, Chapter 5 is concluded with a summary of the findings of this study.

Chapter 2

Background

This chapter provides the theoretical ground needed to understand the remainder of this thesis. First, I discuss the process of classifying brain signals, starting with Section 2.1 on event-related potentials. These are the input to the decoding pipeline, explained in Section 2.2. Moreover, I provide an in-depth explanation of the LDA classifier in Section 2.3, including the popular shrinkage regularization (2.3.1). Block-Toeplitz LDA builds further upon this with additional assumptions and regularization, as explained in Section 2.4.

2.1 Brain Signals and Event-Related Potentials

Electroencephalogram (EEG) is one of the most widely used brain imaging techniques in BCIs, as it records brain activity through electrodes placed on the scalp [19]. A BCI that uses these EEG signals is referred to as an EEG-based BCI. Among the various types of signals that can be extracted from EEG-based BCIs are event-related potentials (ERPs).

After the occurrence of a visual, auditory, or sensory stimulus, the brain response can be observed as a small potential (i.e., change in voltage) in EEG signals, which is denoted as an ERP response [19]. Different components can be analyzed from an ERP response, with P300 as the main component [20], which is a positive potential that occurs about 300 ms after the onset of a stimulus [21]. The duration of the P300 response varies and according to [20], the latency range for auditory stimuli is 250-400 ms for most adults 20-70 years old.

The P300 response is used in different paradigms, such as the auditory oddball paradigm, where different stimuli are presented in a sequence to a subject, of which one stimulus occurs infrequently (the "oddball") [20]. The task is to only attend to the oddball stimulus and as a result, the elicited P300 response to this infrequent target stimulus has a higher amplitude than the P300 responses to the ignored, non-target stimuli [20].

The dataset used for this study is essentially based on this paradigm, which uses ERPs, although there are a number of modifications that make the experimental setup fundamentally different from the oddball paradigm [7]. The dataset and experimental setup are explained in Chapter 3 (Methods).

2.2 The BCI Pipeline

All processing steps from acquiring raw brain signals to producing feedback are described in a so-called BCI pipeline. As mentioned in Chapter 1 (Introduction), there exist multiple frameworks that divide these processing steps into a number of components. For this thesis, I follow the model of [8], which describes a BCI system with five main components: signal acquisition, preprocessing, feature extraction, classification, and device control.

Signal Acquisition

The input of a BCI is recorded brain activity. Raw EEG recordings contain both spatial and temporal information [22]. With N_C (spatial) EEG channels and N_T time points, the raw EEG data is stored in a data matrix $\mathbf{X} \in \mathbb{R}^{N_C \times N_T}$. In ERP-based BCIs, such as the auditory oddball paradigm, the stimulus onset for every ERP is marked. Based on this marker information, the raw EEG data is cut into time segments of L time points, also referred to as *epochs*. Consequently, with N_e epochs and every epoch $\mathbf{e} \in \mathbb{R}^{N_C \times N_L}$, the EEG data are stored as $\mathbf{X} \in \mathbb{R}^{N_e \times N_C \times N_L}$.

Preprocessing

Before features are extracted that are used for classification, the raw EEG signals typically undergo a preprocessing phase, where the desired brain signals are cleaned from noise [23]. Noisy EEG channels that contain irrelevant information are removed through dimension reduction techniques such as principal component analysis and independent component analysis [2]. Other undesirable signals that contaminate brain activity, so-called artifacts, are often also removed [2]. Furthermore, the data are resampled with a lower frequency to obtain a lower number of time points, reducing computational costs [24].

Feature Extraction

During feature extraction, discriminative features are extracted that allow differentiation between task-specific brain states [4]. For instance, features in the auditory oddball paradigm contain the P300 component, which is used to discriminate between target and nontarget stimuli [23]. Features can be extracted from multiple channels and time segments, after which they are reshaped to a single feature vector [2]. Every epoch $\mathbf{e}$ is flattened to a feature vector $\mathbf{e} \in \mathbb{R}^D$, where the spatial features N_C and temporal features N_L are concatenated in $D = N_C \cdot N_L$.

Classification

The aim of the classification is to decode the user's mental state based on the feature vector obtained from the feature extraction phase [23]. In the auditory oddball paradigm, the classifier predicts whether the stimulus was a target or nontarget by assigning a class label to its corresponding feature vector.

Device Control

After classifying the recorded brain signals, the classifier output is used to control an external device [25]. For example, it can be used to control a speller device [26, 27], BCI-based prosthetics [4, 5], and wheelchairs [6]. Furthermore, the output can be used to return feedback to patients in rehabilitation paradigms [4, 5, 7].

2.3 Linear Discriminant Analysis

Linear discriminant analysis (LDA) is a supervised classification method, often used in BCIs [9]. This method makes three assumptions on the data distributions [11]: 1) the features of each class follow a Gaussian distribution, 2) the Gaussian distributions of each class have the same covariance matrix, and 3) the true class distributions are known.

LDA will be explained using a simple case with only two classes. In short, LDA learns from labeled data by fitting a decision boundary hyperplane through the data to optimally separate the different classes [28]. As indicated in the name, LDA linearly projects data points onto a one-dimensional subspace, i.e. a line, perpendicular to the decision boundary. A data point $\mathbf{x} \in \mathbb{R}^D$ is essentially a feature vector of D dimensions and projected onto this subspace according to the following linear function:

$$y = \mathbf{w}^\mathbf{T}\mathbf{x} + b \tag{2.1}$$

with $\mathbf{w}$ as the *weight vector* and b as the *bias* [29]. With $K = 2$ classes and each class denoted as C_k, a novel feature vector $\mathbf{x}$ is then assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise. According to [29], the weight vector $\mathbf{w}$ determines the orientation of the decision hyperplane, whereas the bias b denotes the location of the decision hyperplane.

The classifier parameters $\mathbf{w}$ and b can be derived using Fisher's criterion [29]. Let class C_1 have N_1 data points and class C_2 have N_2 data points. The mean vector for each class is then given by

$$\mathbf{m}_1 = \frac{1}{N_1}\sum_{n \in C_1} \mathbf{x_n} \qquad \mathbf{m}_2 = \frac{1}{N_2}\sum_{n \in C_2} \mathbf{x_n} \tag{2.2}$$

For every class k, the data can be mean-centered by subtracting the class mean $\mathbf{m}_k$ from each data point:

$$\mathbf{X}_k = \begin{bmatrix} \mathbf{x}_1 - \mathbf{m}_k \\ \mathbf{x}_2 - \mathbf{m}_k \\ \vdots \\ \mathbf{x}_{N_k} - \mathbf{m}_k \end{bmatrix} \in \mathbb{R}^{N_k \times D} \tag{2.3}$$

Using the mean-centered data $\mathbf{X}_k \in \mathbb{R}^{N_k \times D}$ for class k, the covariance matrix for each class can be computed with

$$\Sigma_1 = \frac{1}{N_1 - 1}\mathbf{X_1^T X_1} \qquad \Sigma_2 = \frac{1}{N_2 - 1}\mathbf{X_2^T X_2} \tag{2.4}$$

The class covariance matrices Σ_1 and Σ_2 allow the computation of the *between-class* covariance matrix Σ_B and *within-class* covariance matrix Σ_W through

$$\Sigma_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\mathbf{T} \tag{2.5}$$

$$\Sigma_W = \Sigma_1 + \Sigma_2 \tag{2.6}$$

The idea of Fisher was to maximize the between-class covariance while minimizing the within-class covariance [29]. The Fisher's criterion can be written as

$$J(\mathbf{w}) = \frac{\mathbf{w^T}\Sigma_B\mathbf{w}}{\mathbf{w^T}\Sigma_W\mathbf{w}} \tag{2.7}$$

After differentiating (2.7) with respect to the weight vector $\mathbf{w}$ [29], we find that

$$\mathbf{w} \propto \Sigma_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1) \tag{2.8}$$

Therefore, the LDA classifier's parameters $\mathbf{w}$ and $\mathbf{b}$ for binary, balanced classes can be obtained through:

$$\mathbf{w} = \Sigma_W^{-1}(\mathbf{m_2} - \mathbf{m_1}) \tag{2.9}$$

$$b = -\frac{1}{2}\mathbf{w}^T(\mathbf{m_1} + \mathbf{m_2}) \tag{2.10}$$

The definition in (2.10) only holds for balanced classes. In many ERP-based BCIs, such as in the auditory oddball paradigm, the classes are imbalanced, as the stimulus set contains five nontargets and only one target [12]. However, in these scenarios, the bias is irrelevant, because the target stimulus can be determined by selecting the stimulus with the largest output from (2.1) (with the same bias b for all six stimuli, the influence of b is then canceled out).

2.3.1 Shrinkage Regularization

The covariance matrix is essential in the computation of the LDA classifier's parameters. However, for high-dimensional data with only a few data points, a systematic error in the estimation of the covariance matrix is observed [11], which degrades the classification performance of LDA.

To compensate for this systematic error, shrinkage is a popular solution [11]. Let $\hat{\Sigma}$ be the estimated (empirical) covariance matrix obtained from (2.4). The shrinkage covariance matrix then is computed with

$$\widetilde{\Sigma}(\gamma) := (1-\gamma)\,\hat{\Sigma} + \gamma\nu\mathbf{I} \tag{2.11}$$

with $\gamma \in [0, 1]$ as the tuning parameter, $\mathbf{I}$ as the identity matrix, and ν as the average eigenvalue trace (see [11]). The shrinkage parameter $\gamma = 0$ results in the empirical covariance matrix, while $\gamma = 1$ leads to a spherical covariance matrix.

Ledoit and Wolf have found an analytical method to compute the shrinkage parameter [30]. The regularized form of LDA which uses this analytical solution to select the shrinkage parameter is called shrinkage LDA (sLDA).

2.4 Block-Toeplitz LDA

The novel Block-Toeplitz LDA (BT-LDA) further regularizes the shrinkage covariance matrix using domain-specific knowledge of the EEG, and has demonstrated significant improvements compared to sLDA [12].

Note that in this section, I follow the notation of Sosulski and Tangermann [12], where t denotes time points. This differs from my later use of t which represents the trial number. Furthermore, the channels are denoted with the small letter c.

First, the authors of BT-LDA define an epoch $\mathbf{e}$ as a *channel-prime* feature vector. This means that when $\mathbf{e} \in \mathbb{R}^{N_c \times N_t}$ is flattened into $\mathbf{e} \in \mathbb{R}^{N_c N_t}$, all channel features are stacked first:

$$\mathbf{e} := \left[x_{c_1}^{t_1}, x_{c_2}^{t_1}, \ldots, x_{c_{N_C}}^{t_1}, x_{c_1}^{t_2}, \ldots, x_{c_{N_C}}^{t_{N_T}} \right] \tag{2.12}$$

where $x_{c_m}^{t_n}$ is the signal at channel m and time point n of that epoch. Stacking each flattened epoch $\mathbf{e}$ for all N_e epochs, delivers the data matrix $\mathbf{X} \in \mathbb{R}^{N_c N_t \times N_e}$. In contrast to (2.3), the data matrix $\mathbf{X}$ in this section has the number of features in its first dimension and the number of data points (epochs) in the second dimension. Therefore, instead of using Equation (2.4), this section computes the estimated covariance matrix with $\mathbf{\Sigma} = \frac{1}{N_e - 1} \mathbf{X}\mathbf{X}^{\mathbf{T}}$. As a result of N_e channel-prime feature vectors $\mathbf{e}$, the covariance matrix $\mathbf{\Sigma}$ becomes a channel-prime matrix. This matrix can be seen as a block matrix where each block $\mathbf{C}_{i,j} \in \mathbb{R}^{N_c \times N_t}$ is defined as the channel-wise (cross-)covariance matrix between time points i and j.

2.4.1 Regularization

The regularization steps of BT-LDA follow two assumptions for ERP data. The first assumption is that the covariance in the time domain is based only on the temporal distance $|t_i - t_j|$ between two samples. The second assumption is that the covariance between two samples shrinks towards zero as their temporal distance increases.

Following the first assumption, a Block-Toeplitz structure is enforced in the covariance matrix by averaging blocks that share the same temporal distance. Next, the second assumption is implemented through a monotonically decreasing tapering function [31]. For a more detailed explanation of the underlying reasons and implementation of the assumptions, the reader is referred to the original publication by the authors [12].

2.5 BT-LDA in the Pipeline

To ensure BT-LDA performs optimally, the assumptions it makes should not be violated in the pipeline. In this section, three important prerequisites for the data are listed [12].

(1) **The spatio-temporal data matrix X must be channel-prime**.

(2) **The chosen time unit must be constant**. The time unit could be either a time point or time interval, as determined in the feature extraction phase of the pipeline. If time intervals are chosen, then they must be of equal size.

(3) **Baseline correction should be disabled**.

2.5.1 Adaptation in BT-LDA

Updating can take place on different levels in the adaptive BT-LDA classifier. Adaptation strategies can be implemented on the level of i) the data matrix $\mathbf{X}$ and label vector $\mathbf{y}$, ii) the class means and covariance matrix, and iii) the weight vector $\mathbf{w}$ and bias b, which are the classifier's parameters. For updating at level ii), both the class-specific and the pooled version (combining the classes) of the mean and covariance matrix can be considered.

For adaptation strategies on BT-LDA, such as a convex combination that combines two classifiers, both classifiers must have completed all regularization steps. This means that both the first and the second classifier must have had shrinkage regularization, enforcement of the Toeplitz structure, and linear tapering.

Chapter 3

Methods

The following chapter is presented in five sections. Section 3.1 contains information about the dataset used for this study. Section 3.2 provides details on how this data is classified in the decoding pipeline. Related work on adaptive classifiers is discussed in Section 3.3, after which four new versions of BT-LDA are proposed in Section 3.4, of which two use a within-session adaptation strategy. Lastly, in Section 3.5, I define the evaluation protocol used to examine the influence of the proposed adaptation strategies.

3.1 Dataset

3.1.1 Participants

The dataset used for this study has been obtained from an auditory aphasia rehabilitation paradigm by Musso et al. [7]. The data has been collected from 10 patients suffering from chronic aphasia caused by an ischaemic stroke on the left hemisphere. All subjects were right-handed, native German speakers with no prior experience with auditory brain-computer interface (BCI) paradigms.

3.1.2 Experimental protocol

The task of the patients was to listen to a sentence in which the last word was missing. After that, a sequence of six different words was played, of which one was supposed to be the missing last word of the sentence. The sequence of six words was repeated multiple times, varying in the order of words per sequence. The patient had to attend to the correct word, which will be denoted as the *target* word, while ignoring the other five incorrect words, which are called *nontargets*.

The described task with a single sentence followed by a number of *iterations* (sequences) of six words, is called a trial. A single word played on a speaker is also referred to as a *stimulus*. Thus, a single trial consists of multiple iterations, each containing six stimuli. For every stimulus, an *epoch* (time segment) has been extracted from the brain recordings (as explained in Section 2.2). Important to mention is that in the literature, the word *trial* is used ambiguously. In this study, a trial consists of multiple stimuli and, therefore, multiple epochs. A single trial took between 35–60 s.

At the end of a trial, typically consisting of 15 iterations, the classifier predicted the target word of that trial. In the case of 15 iterations, this would mean that a single trial contained 90 stimuli. Thus, 90 epochs, which were the trial's selected six words, played 15 times each.

For each stimulus, the classifier predicted the distance from that word to the decision boundary (see Equation 2.1). This is a numeric value obtained in the one but last step of an LDA classifier. The final step of LDA is to assign the stimulus to a class based on the sign of this numeric value (whether it is positive or negative), but this step was not included in the protocol. After a complete trial of 90 stimuli, 90 of these distances to the decision boundary were obtained, which are 15 per word. For each word, the average of these distances was computed. The word with the highest average was determined as the final decoded word of that trial. This classification procedure is shown in Figure 3.2E.

Depending on whether the decoded target word was correct and how well the classifier could decode it, the patient received feedback at the end of the trial. This feedback reinforced the patient to find optimal strategies for performing the task.

To obtain a general indication of classification performance that did not depend on the selected target word, six trials were combined into a single run, with each trial containing a different target word. After a run, the patient could take a break. In a single session, multiple runs were recorded, with six runs forming a single block (Figure 3.1A). A single session (Figure 3.1B) consists of multiple blocks, of which the final block may contain less than six runs. The number of sessions differed per subject. Nevertheless, for each subject, all sessions combined lasted around 30 hours.

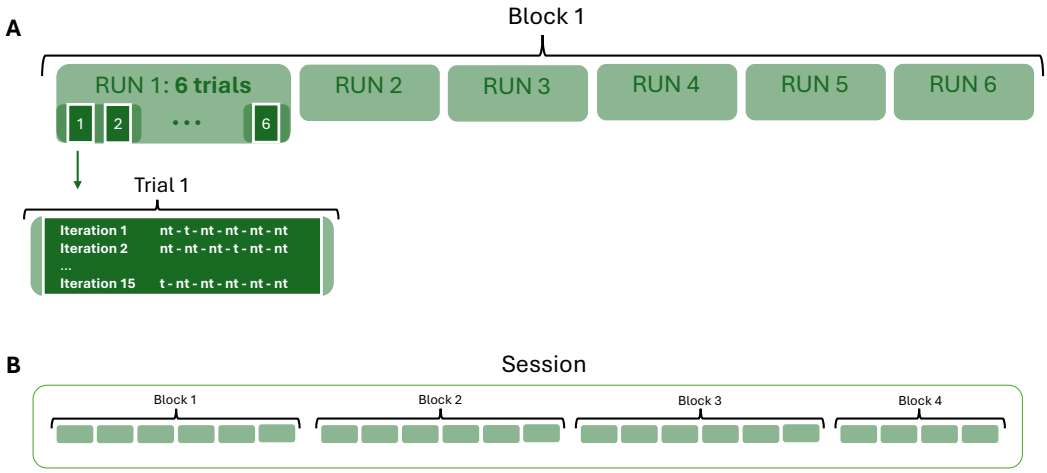


Figure 3.1: (**A**) The typical structure of a single block where t = target; nt = nontarget. (**B**) The typical structure of a session. The last block of a session may contain less than six runs.

3.1.3 Offline and online sessions

For every patient, the first two sessions and the last session are calibration (i.e., offline) sessions. This means that in all trials, the patient was explicitly told to attend to a specific target word, and no feedback was given. The data from the first two offline sessions was used to train the initial classifier. An exception is patient 8, for which the third session was also an offline session and, therefore, used for training. For all patients, the last recorded session was also an offline session, but this is not relevant for our purposes and therefore ignored. The remaining sessions are online sessions, where in each trial, the patient performed the task and received feedback.

3.1.4 Classifier in the protocol

The classifier that was used in the language rehabilitation experiment was an adaptive shrinkage LDA. The classifier was pre-trained on the first two calibration sessions to classify trials in the third (online) session. In all subsequent online sessions, the classifier was trained on the data of the previous session using transfer learning.

Within an online session, the classifier was continuously updated using the supervised adaptation strategy in [32]. This strategy updated the class means (Equation 2.2) and the inverse of the within-class covariance matrix (Equation 2.6) through a convex combination of the current classifier with a novel feature vector $\mathbf{x}$, obtained from every epoch. Thus, within a single trial, the class means and the inverse of the covariance matrix were updated and stored after each epoch. However, only after the trial had finished (i.e., the classifier had predicted the target word) were the (final) updated parameters used to replace the classifier. This classifier will be used as a second baseline in this thesis, and will be denoted as *Adaptive CC sLDA*.

3.1.5 Conditions

In the experimental setup, multiple conditions were used, varying the number of loudspeakers and stimulus onset asynchrony (SOA). For the number of speakers, two conditions were used. There was a 6D condition, where the patient was surrounded by six loudspeakers on which each of the six words was played, and a headphone (HP) condition, where the patient only heard the words from two speakers. Note that the HP condition made the task considerably difficult since the spatial cues, as obtained from the 6D condition, were removed. For the SOA, there were also two conditions: either an SOA of 350 ms or 250 ms was used. Also, here, the latter condition made the task harder.

In the calibration sessions, multiple runs were recorded for every possible combination of conditions. That is, "6D 350", "6D 250", "HP 350", and "HP 250". Depending on how well the classifier was able to classify the calibration sessions, the first online session (i.e., the third session) started with either the "6D 350" or "6D 250" condition.

The described setup is the general protocol, but in some sessions an exception was made with a different condition from those described above. All used conditions, including these exceptions, are listed in Appendix A.

3.1.6 Dynamic stopping

The dynamic stopping method by Höhne [27] was used to speed up the classification of a trial if the classifier was confident enough about the target word. This means that in a trial where dynamic stopping was used, the number of iterations is lower than 15, thus making the total amount of stimuli lower than 90. When dynamic stopping occurred, the patient received a more positive feedback (instead of the standard positive feedback).

3.2 Pipeline Design

This section describes the pipeline that I used for the online simulation. The preprocessing and feature extraction steps are the same for all four proposed versions of BT-LDA and the *Adaptive CC sLDA*. The configurations in this design differ from those used during the original aphasia experiment. The reason behind this is that the implementation of Block-Toeplitz LDA imposes some restrictions, as mentioned in Section 2.5. Furthermore,

to keep the evaluation of the adaptation strategies as pure as possible, some preprocessing steps (e.g., epoch rejection) were left out.

3.2.1 Data preprocessing

For the preprocessing phase, the MNE-Python library was used [33]. The data were recorded using 69 (63 EEG) / 37 (31 EEG) channels for the calibration / online sessions. First, only EEG channels were selected, and the additional channels of the calibration sessions were discarded so that the raw data always contained 31 EEG channels. The next step was to use an infinite impulse response (IIR) filter with a bandpass range of [0.5 Hz, 16 Hz]. Then, the originally 1000 Hz raw data were resampled to 100 Hz, using a decimate of 10. After downsampling, the raw data were cut into epochs, each with a time window of -0.2 s before and 1 s after the onset of a stimulus. As a result, every epoch contained 120 time points per channel. No baseline correction was used, as this should be avoided with BT-LDA (see [12] and 2.5).

Next, all epochs were concatenated and organized into trials as follows. First, every six epochs were grouped into a single iteration. Then, based on the marker information, the number of iterations Z in a trial was determined. Z could be any number between 8 and 15 (both included). Using this information, Z iterations were combined into a single trial, which was stored in a list called `trials`. It should be emphasized that Z is not a constant number and differs per trial. The preprocessing steps are shown in Figure 3.2A.

3.2.2 Feature extraction

In the feature extraction phase, the trials are processed and reshaped to a 2D matrix $\mathbf{X}$ and a 1D vector $\mathbf{y}$, which can be used as input for the classifier. After all preprocessing steps, a list of trials is obtained. Note that `trials` is a nested list of depth four, with an inhomogeneous shape on the second dimension, due to the varying size Z for the number of iterations per trial. As a result of this inconsistent structure, the trials cannot be represented as a regular tensor.

Nevertheless, the features are extracted on the epoch level. Therefore, iterating over every epoch, extracting features of that epoch, and concatenating all epochs in the end would still result in our desired data matrix $\mathbf{X}$ and labels $\mathbf{y}$.

For every epoch, the following feature extraction steps are taken. First, the time points are averaged over predefined time intervals, which range to 800 ms to capture the ERP components N400 and P600, which are related to language [7]. Based on a 4-fold cross-validation performed on sessions 1 and 2 across all patients, 14 time intervals ranging from 0.1 s to 0.8 s, incremented in 50 ms steps, were selected. The results of this selection can be found in Appendix B.1.

Originally, with 31 EEG channels and 120 time points per channel (explained in 3.2.1), every epoch $\mathbf{e}$ is structured as:

$$\mathbf{e} \in \mathbb{R}^{31 \times 120} \tag{3.1}$$

After averaging the time points over the 14 predefined time intervals, we obtain the following structure per epoch:

$$\mathbf{e} \in \mathbb{R}^{31 \times 14} \tag{3.2}$$

Next, the data must be structured channel-prime, as required for BT-LDA (see [12] and 2.5). Therefore, the dimensions of an epoch are swapped:

$$\mathbf{e} \in \mathbb{R}^{14\times 31} \tag{3.3}$$

Lastly, the features are flattened to reshape the epoch into a single, channel-prime, 1D vector:

$$\mathbf{e} \in \mathbb{R}^{434\times 1} \tag{3.4}$$

Finishing this procedure on all epochs allows us to stack the final 1D epochs into a 2D matrix $\mathbf{X} \in \mathbb{R}^{N\times D}$, where N is the number of epochs and D is the number of features (434). The labels are stored in a 1D vector $\mathbf{y} \in \mathbb{R}^{N\times 1}$. These steps are also described in Figure 3.2C.

3.2.3 Classification

As explained in Section 3.1.2, the classifier predicts the target word for every trial and the number of iterations Z per trial is not constant due to dynamic stopping (3.1.6). In the following example, we assume that a trial contains 15 iterations.

A trial with 15 iterations contains 90 epochs, which are obtained from the trial's selected six words, each repeated 15 times. For every epoch $\mathbf{e} \in \mathbb{R}^{343\times 1}$ in the trial, the classifier predicts the distance to the decision boundary through its decision function.

Let the unique word ID for each of the six words be

$$s \in \{1, 2, 3, 4, 5, 6\}$$

For each word, the distance $d_{\mathbf{e}}$ to the decision boundary in iteration i is obtained with

$$i \in \{1, 2, ..., Z\} = \{1, 2, ..., 15\}$$

$$d_{\mathbf{e}} = \texttt{decision_function(e)}$$

When $d_{\mathbf{e}}$ is computed, the word s is obtained from the marker information and $d_{\mathbf{e}}$ is stored as follows:

$$\texttt{stim_distances[}s\texttt{, }i\texttt{] = }d_{\mathbf{e}}$$

After storing $d_{\mathbf{e}}$ in `stim_distances` for every epoch, the average of every word s over all iterations is computed:

$$r_s^t = \frac{1}{15}\sum_{i=1}^{15} \texttt{stim_distances[s,i]}$$

with t being the trial. Next, the word s with the largest r_s^t score is determined as the target word of that trial.

In summary, in a single trial, the classifier predicts at two levels: i) $d_{\mathbf{e}}$: the distance of an epoch to the decision boundary, and ii) s_t: the predicted word of a trial t. This classification procedure is also shown in Figure 3.2E.

In all proposed versions of BT-LDA and the Adaptive CC sLDA (Section 3.4), the classifier is trained differently. The common ground, however, is the initial classifier in the first online session, which is trained on the offline (calibration) sessions. Then, in the online sessions where these classifiers are deployed, between-session and additional within-session adaptation can take place, depending on the updating strategy.

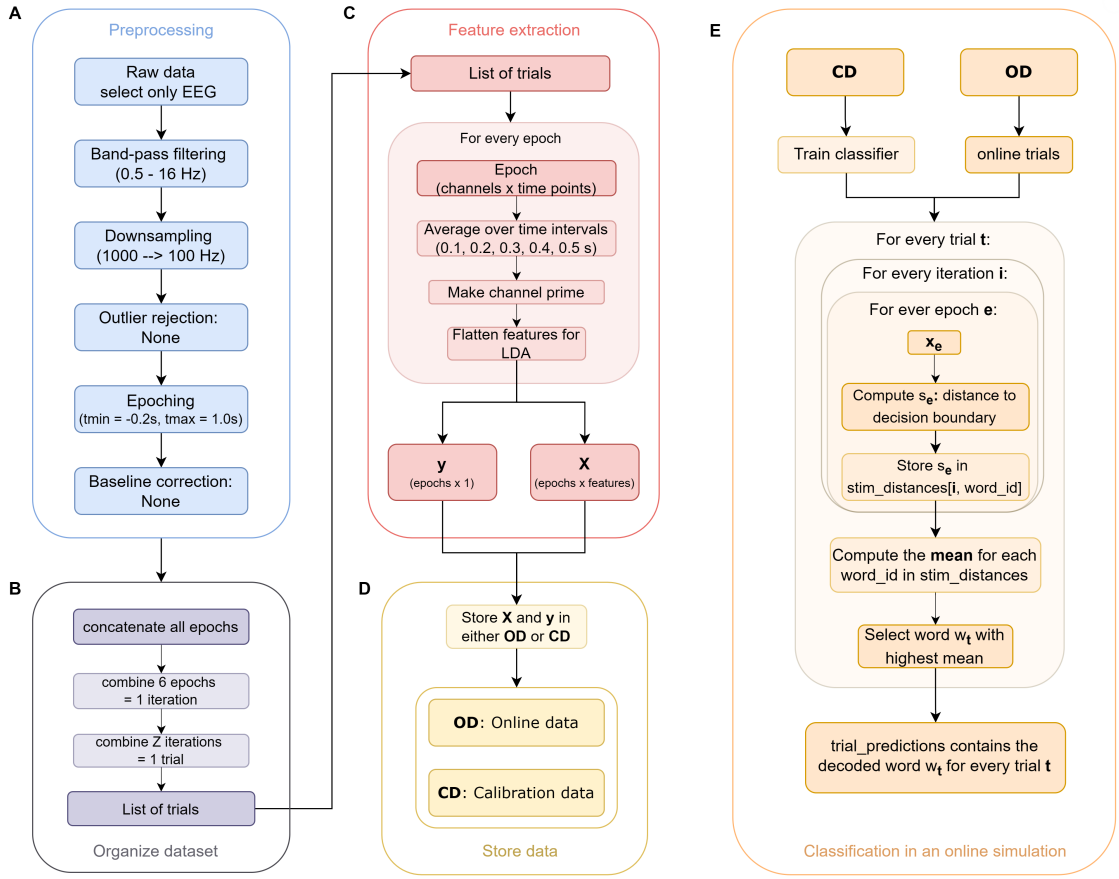


Figure 3.2: The decoding pipeline for a single session. (**A**) Preprocessing. (**B**) Organizing epochs into trials based on the marker information. (**C**) Feature extraction. (**D**) Storing the data. (**E**) Classification procedure for every trial in the session. Steps A, B, C, and D are the same for all four proposed BT-LDA versions. Step E is different: the calibration data CD varies, and the classifier is either trained once or retrained continuously if a within-session updating strategy is used.

3.3 Related Work on Adaptive Classifiers

Lotte et al. [9] provided a review of classification approaches from 2007 until 2017, discussing adaptive classifiers as one of the main approaches of the decade. As briefly mentioned in Chapter 1, I will consider supervised adaptation strategies for BT-LDA. Multiple studies have shown that the adaptive version of LDA surpasses the classification performance of the static version [18, 32, 34, 14].

Previous studies on motor imagery-based BCIs have demonstrated that adaptive classifiers in BCI control, improve the learning curves of both users with poor and good control [32, 34].

However, the dataset for this study is based on ERP features, which is essentially different from oscillatory features. In the oscillatory domain, a trial typically lasts a few seconds, for which a single epoch is extracted from raw EEG recordings and analyzed. In contrast, a trial in ERP-based BCIs may contain multiple epochs that cover a shorter time window, depending on the relevant ERP components expected in the response. For example, in the dataset used for this thesis, a trial lasted between 35-60 s, of which 48-90

epochs were extracted, each covering 1.2 seconds around the stimulus onset.

Recently, Wu et al. [34] showed in both an offline (15 subjects) and online (20 subjects) study, that compared to its static counterpart, an adaptive LDA that uses the convex combination strategy enhanced BCI control when decoding imagined syllables. However, the study was conducted exclusively on healthy participants, leaving the effect of adaptation on patients, for whom the BCI is designed in the first place, unexplored.

All the discussed work on adaptation strategies have been used for either standard LDA or sLDA, but none have been applied on BT-LDA. Therefore, this thesis focuses on the potential improvements that updating strategies could have on the adaptive BT-LDA.

3.4 Proposed Adaptation Strategies

In this thesis, four different versions of BT-LDA are constructed, varying in the usage of transfer learning and any additional adaptation strategies. Additionally, the adaptive sLDA classifier used in the aphasia experiment is implemented (3.1.4). With the terms *between-session* and *within-session* adaptation, I refer to the sessions of a single subject. All constructed classifiers are the following:

(1) **Static Fixed BT-LDA**: This version does not contain any within-session or between-session adaptation. Static-Fixed BT-LDA is only trained once on the initial calibration data. This version of BT-LDA is used as an additional comparison to examine the effect of transfer learning in Transfer Fixed BT-LDA.

(2) **Transfer Fixed BT-LDA**: This version only applies between-session adaptation through transfer learning. This means that for every new session, the previous session is used to train the classifier. No within-session adaptation takes place, meaning that after this classifier is trained for a new session, the classifier remains fixed until the whole session is completed. Transfer Fixed BT-LDA is used as a baseline to compare the additional effects of adaptation strategies.

(3) **Adaptive Window BT-LDA**: This version uses a similar between-session adaptation approach as Transfer Fixed BT-LDA, with the additional implementation of a sliding window strategy for within-session adaptation, which is explained in 3.4.1.

(4) **Adaptive CC BT-LDA**: This version uses both between-session and within-session adaptation, following a strategy that combines the previous and current classifier through a convex combination. This is explained in 3.4.2.

(5) **Adaptive CC sLDA**: This classifier is similar to the classifier used in the original aphasia protocol (section 3.1.4). Note that the preprocessing and feature extraction configurations for this classifier (defined in 3.2) are different from the original classifier in the experiment. Adaptive CC sLDA is used as a second baseline.

3.4.1 Adaptive Window BT-LDA

At the start of every new session, the Adaptive Window BT-LDA is initialized using data of the previous session, following a similar between-session adaptation as Transfer Fixed BT-LDA. However, in contrast to Transfer Fixed BT-LDA, not the whole session is used, but a dataset of predefined window size M.

In addition to the between-session adaptation, within-session updating takes place. The within-session adaptation strategy stores the training set and continuously updates this dataset after every classified trial t in an online session.

With M being the number of trials used for calibration and current trial t_i (with i as the trial number and $i > M$), a classifier is trained on trials t_{i-M} until and including t_i. This classifier is used to classify the next trial t_{i+1}. After this classification, trial t_{i+1} is added to the training set, and at the same time, the oldest trial is removed from the training set. Thus, the new calibration dataset consists of trials t_{i+1-M} until and including t_{i+1}. This dataset is then used to train a new BT-LDA classifier, which is going to classify the next trial.

The optimal window size M is found by using all data of a representative patient with not the best, neither the worst performance. The epoch-wise accuracy (section 3.5) of the Transfer Fixed baseline is computed for different sizes of M and the saturation point over these sizes is found, i.e. the value of M where the accuracy does not appear to increase substantially.

3.4.2 Adaptive CC BT-LDA

For the Adaptive CC BT-LDA that uses the convex combination strategy, the first two offline sessions are used to train the initial classifier. In the third session, i.e., the first online session, the trained classifier is deployed and continuously updated within that session. At the end of the third session, the final updated classifier is obtained and taken into the fourth session, where further updating takes place. This procedure holds for all following online sessions.

Note that transfer learning is used here, but in contrast to Transfer Fixed BT-LDA and Adaptive Window BT-LDA, this version does not explicitly use the calibration data from the previous session. Instead, the classifier's parameters are used, which is done as follows.

BT-LDA is a regularized version of LDA. As explained in Chapter 2, the data parameters of LDA are computed through

$$\mathbf{m}_c = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{x}_i \tag{3.5}$$

$$\Sigma_C = \frac{1}{N_c - 1} (\mathbf{X}_c - \mathbf{m}_c)(\mathbf{X}_c - \mathbf{m}_c)^{\mathbf{T}} \tag{3.6}$$

with $c \in \{1, 2\}$ (1: target, 2: nontarget) as the class; N_c as the number of samples per class; $\mathbf{x}_i$ as data point i; and $\mathbf{m}_c$ as the mean of class c; $\mathbf{X}_c$ as the data matrix of class c (see 3.2.2) and Σ_c as the covariance matrix of class c.

The regularized Block-Toeplitz covariance matrix is obtained after the following regularization steps [12]

$$\Sigma_{C_shrinkage} = leidot_wolf_shrinkage(\Sigma_c) \tag{3.7}$$

$$\Sigma_{C_toep} = average_block_diagonals(\Sigma_{C_shrinkage}) \tag{3.8}$$

$$\dot{\Sigma}_C = linear_tapering(\Sigma_{C_toep}) \tag{3.9}$$

Since the two classes (target and nontarget) are imbalanced, the pooled covariance matrix is obtained with the following formula

$$\dot{\Sigma}_W = (\frac{1}{6}\dot{\Sigma}_1 + \frac{5}{6}\dot{\Sigma}_2) \tag{3.10}$$

Which gives the final classifier parameters:

$$\mathbf{w} = \dot{\Sigma}_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1) \tag{3.11}$$

$$b = -\mathbf{w}^{\mathbf{T}}(\frac{1}{6}\mathbf{m}_1 + \frac{5}{6}\mathbf{m}_2) \tag{3.12}$$

Adaptive CC BT-LDA updates the classifier within a session by combining the old classifier with a new classifier. After completing a new trial t, a new BT-LDA classifier is trained on this single trial. This means that the BT-LDA is trained on a data matrix $\mathbf{X}_t \in \mathbb{R}^{N\times 248}$ with N being the number of epochs in that trial. N can be any number between 48 and 90, in steps of 6 (the reason for this is explained in 3.1.6).

The updated class means and class covariance matrices are obtained through a convex combination of the old classifier's parameters with the new classifier's parameters. With a new trial t, let $\mu_c(t-1)$ be the old mean (Equation (3.5)) and $\dot{\Sigma}_c(t-1)$ the old covariance matrix (Equation (3.9)) for class c. These are the old classifier's parameters. The new classifier's parameters are obtained through Equations (3.5)–(3.9) and are now denoted as $\mathbf{m}_{c_trial}$, $\dot{\Sigma}_{c_trial}$ for the class mean and class covariance matrix of trial t respectively. Then, the updated parameters are obtained using the following formula:

$$\mu_c(t) = (1 - UC) \cdot \mu_c(t-1) + UC \cdot \mathbf{m}_{c_trial} \tag{3.13}$$

$$\dot{\Sigma}_c(t) = (1 - UC) \cdot \dot{\Sigma}_c(t-1) + UC \cdot \dot{\Sigma}_{c_trial} \tag{3.14}$$

With UC being the update coefficient, $\mu_c(t)$ the updated mean, and $\Sigma_c(t)$ the updated covariance matrix for the updated classifier. The classifier is then trained following Equations (3.11) and (3.12).

3.4.3 Optimizing the UC-values for Adaptive CC BT-LDA

Before deploying the Adaptive CC BT-LDA, the UC-values (UC_μ and UC_Σ) have to be determined beforehand. This UC-pair can be either general, i.e., the same for all patients, or subject-specific. For this thesis, a general UC-pair is determined. Furthermore, the UC-values are ideally separated, as previous work showed that the optimal UC_μ and UC_Σ can have different orders of magnitude [34].

To find the optimal UC-value range, I first used the data of the representative patient with neither the best nor the worst performance with the baseline Transfer Fixed BT-LDA. I performed a a grid search over all values of the UC-pair, using the following range: $UC_\mu, UC_\Sigma \in \{0.5 \cdot 2^{-13}, 0.5 \cdot 2^{-12}, \ldots, 0.5 \cdot 2^{1}\}$ (15 values in total).

For each UC-pair, an online simulation was conducted over all sessions of the representative patient. The average epoch-wise and trial-wise score was computed across all sessions and stored in a 15x15 matrix for both metrics. These results were used to reduce the range for both UC_μ and UC_Σ.

After reducing the range to less than 15 values, the optimal UC was found by following the similar procedure, but this time using the data of multiple patients. To reduce the risk of overfitting, the data of patients 1, 3, 5, 7, and 9 were used to determine the optimal UC-pair for the remaining patients and vice versa. This means that two potentially different general UC-pairs are obtained.

3.4.4 Additional variability in the online simulations

During the experimental setup of the language therapy, the conditions (defined in 3.1.5) were changed depending on the classification performance of the classifier. For instance, (online) sessions 3 to 10 of patient three were recorded using the "6D 250" setup. In session 11 and beyond, the experimenters changed the setup to the "HP 250" condition.

When facing a session with a new condition, there are two ways to train the initial classifier. The first approach is to use the runs with this new condition that were recorded during the first calibration sessions. This was done in the original protocol of the dataset [7]. The second approach is to keep using the previous session or classifier despite the difference in conditions. Which approach is used differs per classifier. For the Static Fixed BT-LDA and Adaptive CC sLDA, the first approach was used. For the Transfer Fixed, Adaptive Window, and Adaptive CC BT-LDA, the second approach was used.

Another challenge, although less frequent, was a change in condition within a session. While it is possible to split such a session into multiple parts, separating the different conditions, I decided to keep the session as a whole instead and document where the conditions had changed.

Furthermore, and importantly, the structure of the data is not constant across sessions and patients. There is variability at different levels. The amount of iterations per trial differs due to dynamic stopping in the original protocol. This means that a trial may contain any number of epochs between 48 and 90, in steps of 6. Furthermore, while the number of trials per run is always six, the amount of runs per session differs.

This variability in structure results in sessions with a considerable variation in the number of epochs (data points), both between subjects and within subjects. For example, session 6 of patient 2 contains 10188 epochs, while session 7 contains 7320 epochs. Session 6 of patient 3 contains 5238 epochs.

Lastly, the number of sessions differs per patient. For patients 2 and 5, 13 online sessions were recorded (the least amount), while for patient 7, 27 sessions were recorded (the most amount).

3.5 Evaluation

3.5.1 Calibration sessions

As mentioned in section 3.1.3, the initial two sessions were offline sessions, which were used to train the classifier at the start for all five classifiers. Two performance metrics were selected to compare the different versions: the *epoch-wise* and *trial-wise* accuracy. These scores are computed at the end of an online simulation. For every patient, all recorded online sessions were used to conduct an online simulation. The protocol for the baseline Transfer Fixed BT-LDA, is shown in Figure 3.3.

For the epoch-wise accuracy, the AUC-ROC (Area Under the Receiver Operating Characteristic curve) is computed at the end of a session. This metric is independent of the threshold chosen for the bias b parameter of the BT-LDA classifier.

The trial-wise accuracy is computed as the ratio of correctly predicted trials:

$$\frac{correct\ trial\ predictions}{total\ trial\ predictions}$$

How these two scores are computed in an online simulation is shown in Figure 3.4.

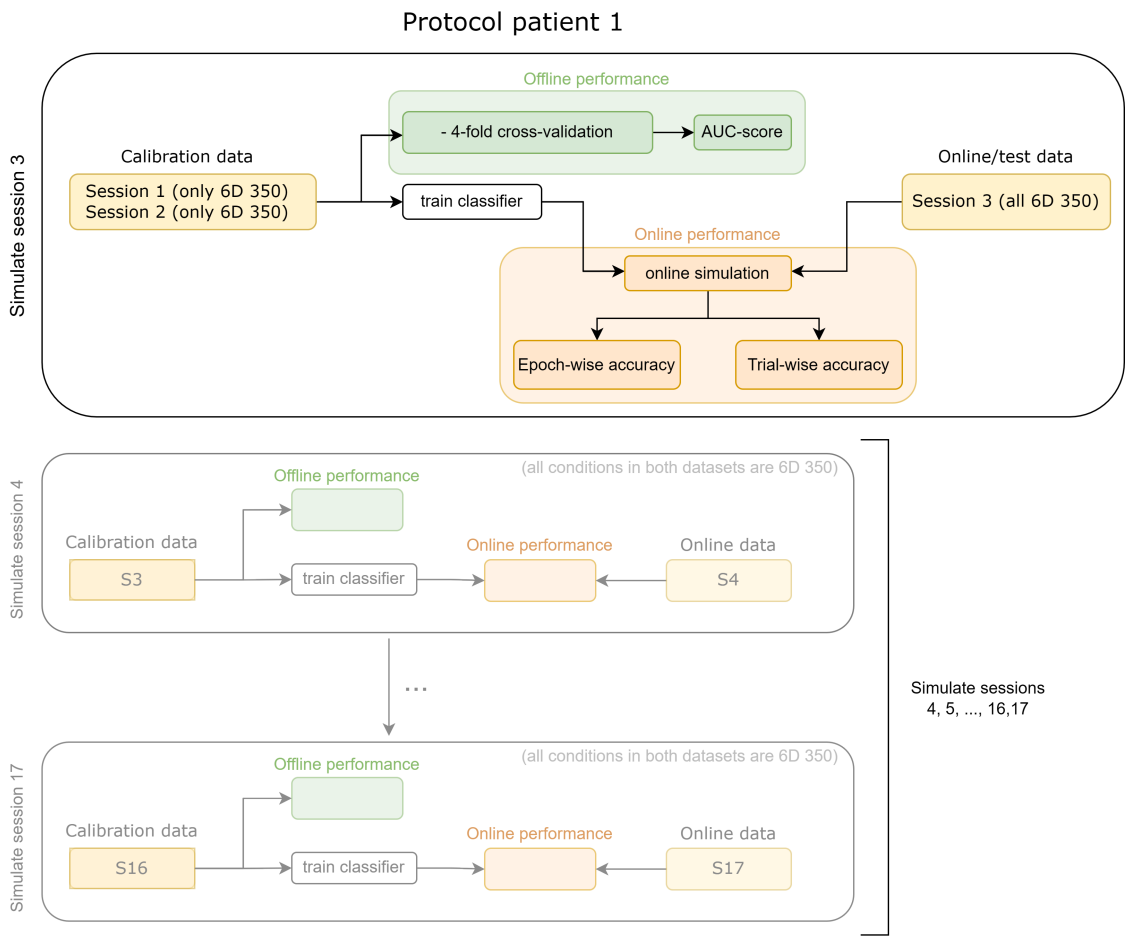


Figure 3.3: Evaluation protocol for Transfer Fixed BT-LDA on the data of patient 1. For every simulated online session, the classifier is trained on the previous session using transfer learning (with the exception of the first online session, which uses sessions 1 and 2). The offline performance of BT-LDA on the calibration data is computed using a 4-fold cross-validation. Then, the "online" data is used to simulate an online session, after which an epoch-wise and trial-wise accuracy is obtained.

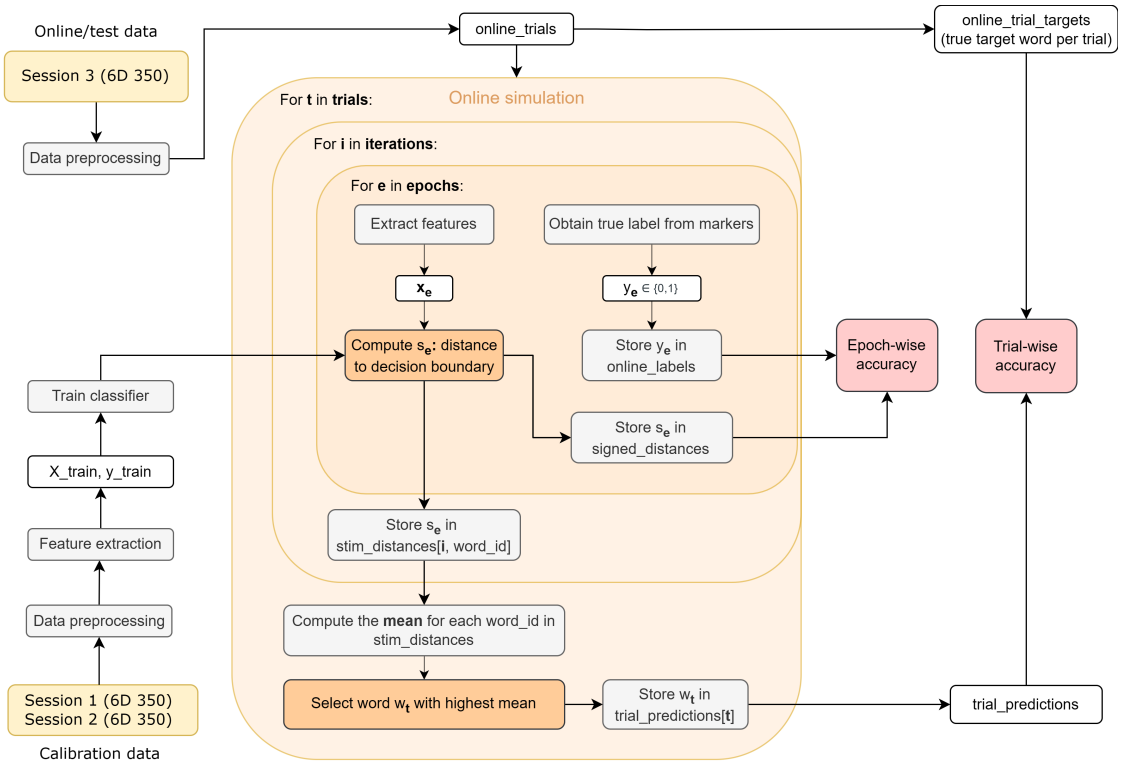


Figure 3.4: Classification and evaluation of an online simulated session. In this example, the classifier is trained on sessions 1 and 2. The data of session 3 is used to simulate an online session. Inside an online session, the classifier predicts at two levels: i) $s_\mathbf{e}$: the distance of an epoch to the decision boundary, and ii) $w_\mathbf{t}$: the predicted word of a trial. The performance is assessed by means of an epoch-wise (level i) and trial-wise (level ii) accuracy.

3.5.2 Statistical Analysis

To test whether the static and two adaptive versions of BT-LDA are significantly different from the baselines Transfer Fixed BT-LDA and Adaptive CC sLDA, a statistical analysis is required. For each baseline, I have chosen to create pairs between all classifiers and this baseline, and test for significance using the Wilcoxon signed rank test. This test is a paired, non-parametric test, suitable for the data of this thesis since the performances do not follow a normal distribution (the minimum accuracy score is 0, and the maximum is 1). Furthermore, a paired test is required because the accuracies of two classifiers on the same data depend on each other. To correct for multiple testing a Bonferroni correction was used.

Chapter 4

Results

The results from the experimental design described in the previous chapter are presented in this chapter, followed by an interpretation in the next chapter. The findings are presented in chronological order. First, the classification performance of the *Adaptive CC BT-LDA* using different UC-pairs is presented, after which an optimal UC-pair is selected. Second, the classification performance of all classifiers in an online simulation are compared against the baselines *Transfer Fixed BT-LDA* and *Adaptive CC sLDA*.

4.1 Hyperparameters

4.1.1 Time Intervals

4.1.2 Moving Window Size for Adaptive Window BT-LDA

4.2 Grid search on UC-pairs for Adaptive CC BT-LDA

In this study, the optimal UC-pair is found by performing a grid search on all combinations of UC_μ and UC_Σ. Due to time restrictions, a representative patient was first used to pilot on the UC-value ranges. Based on the learning curves of all patients with the baseline *Transfer Fixed BT-LDA* (see Appendix C), patient 9 was selected for this pilot.

4.2.1 Representative patient to reduce UC-ranges

A grid search was performed on all UC-pairs, using the following range of 15 values: $UC_\mu, UC_\Sigma \in \{0.5 \cdot 2^{-13}, 0.5 \cdot 2^{-12}, \ldots, 0.5 \cdot 2^{1}\}$. The outcome is presented in Figure 4.1.

Moreover, the researchers had changed the experimental conditions of patient 9, switching from the 6D loudspeaker setup to the HP (two loudspeakers) condition at session 11. To further analyze the influence of the UC-values, the data of this patient was split at session 11, separating the two conditions. A new grid search was then performed on both datasets. The results of this analysis are shown in Figure 4.2.

4.2.2 Optimizing the general UC-pair

Based on the results of the representative patient, the following ranges were selected: $UC_\mu \in \{0.5 \cdot 2^{-7}, 0.5 \cdot 2^{-6}, 0.5 \cdot 2^{-5}, 0.5 \cdot 2^{-4}\}$ and $UC_\Sigma \in \{0.5 \cdot 2^{-17}, 0.5 \cdot 2^{-16}, \ldots, 0.5 \cdot 2^{1}\}$. With the new ranges, the optimal general UC-pair was found using the data of multiple patients. The data of (even) patients 2, 4, 6, 8, and 10 were used to determine the optimal UC-pair for (odd) patients 1, 3, 5, 7, and 9, which is shown in Figure 4.3. Similarly, the

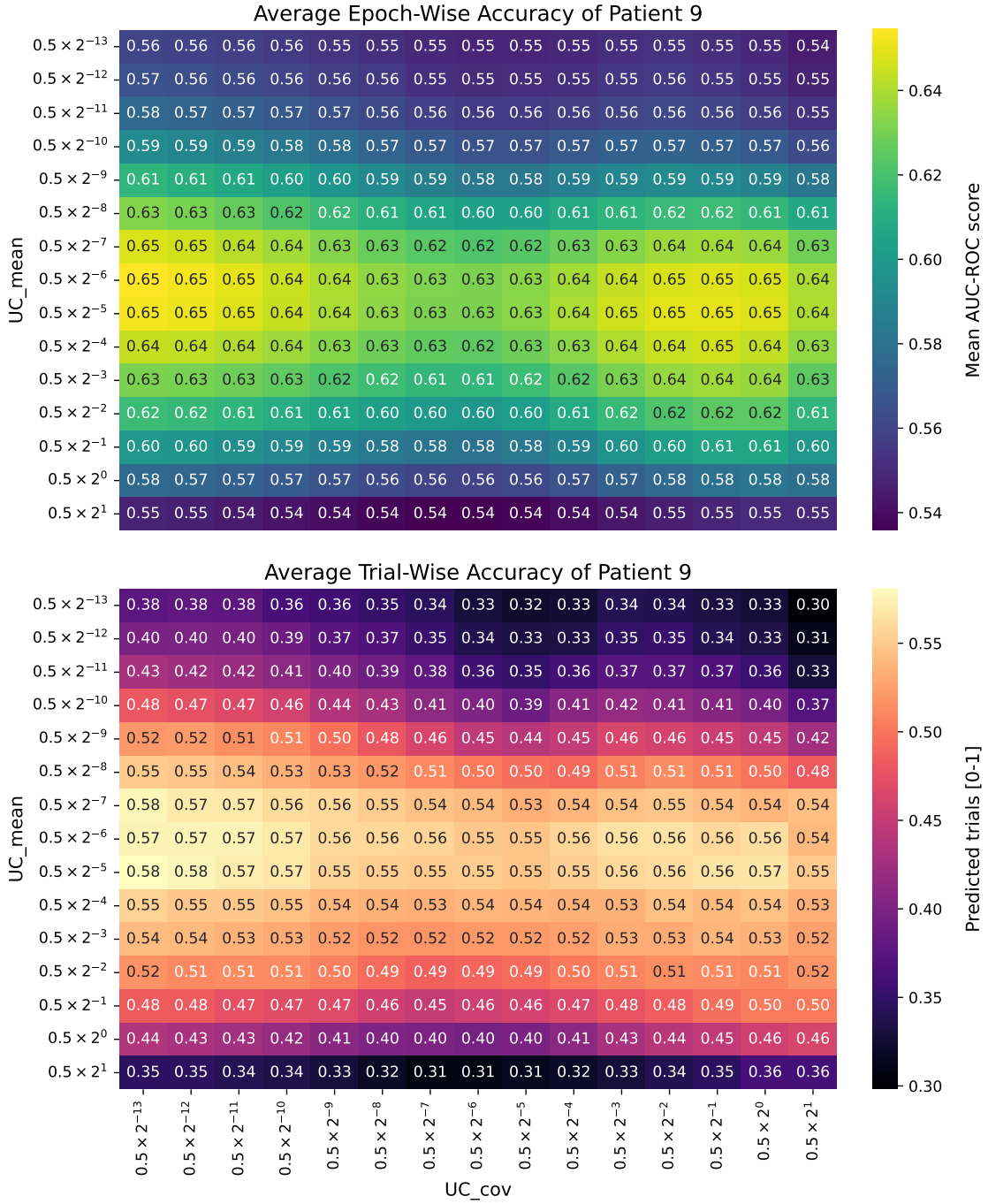


Figure 4.1: Results of a grid search on UC-pairs on patient 9. For each UC-pair, an online simulation was performed and the average score over all sessions was computed. **Top:** Epoch-wise accuracy. **Bottom:** Trial-wise accuracy. Based on these results, the following UC-ranges have been selected: $UC_\mu \in \{0.5 \cdot 2^{-7}, 0.5 \cdot 2^{-6}, 0.5 \cdot 2^{-5}, 0.5 \cdot 2^{-4}\}$ and $UC_\Sigma \in \{0.5 \cdot 2^{-17}, 0.5 \cdot 2^{-16}, \ldots, 0.5 \cdot 2^{1}\}$.

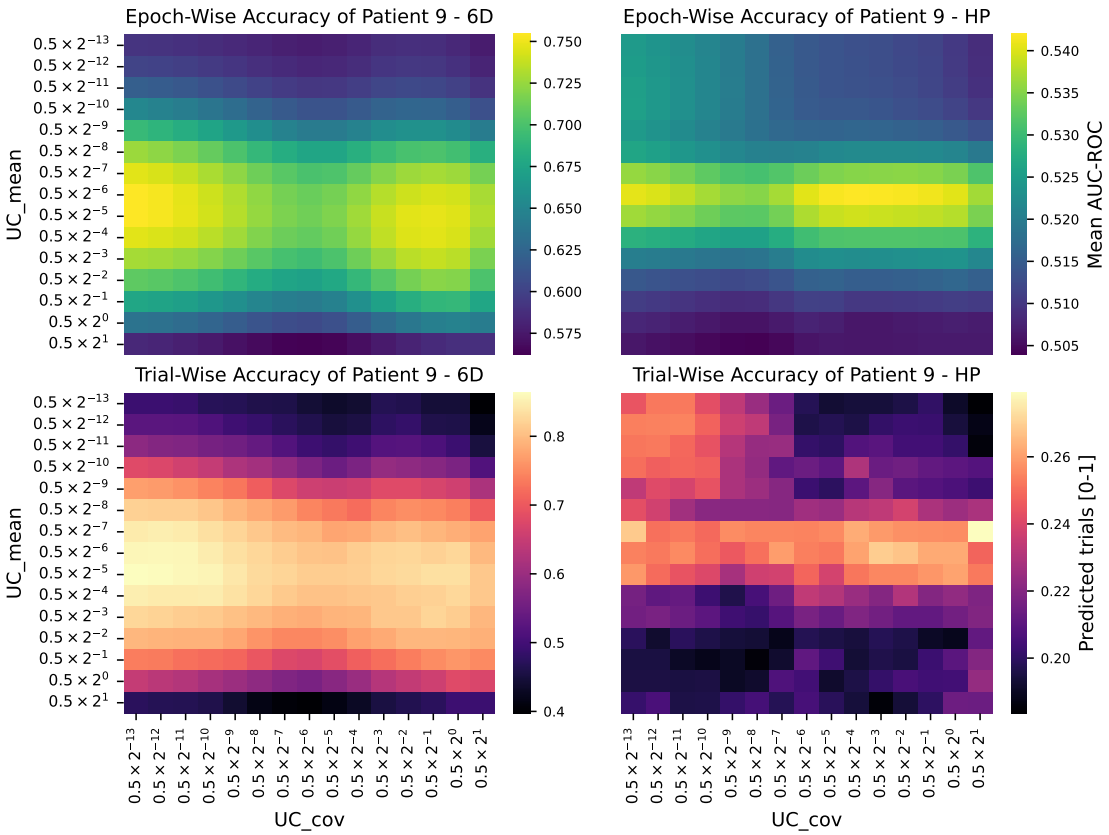


Figure 4.2: Heatmaps of epoch-wise (top) and trial-wise (bottom) accuracy for the 6D (left) and HP condition (right), using the data of patient 9.

optimal UC-pair for the even patients has been determined from the results of the odd patients, shown in Figure 4.4.

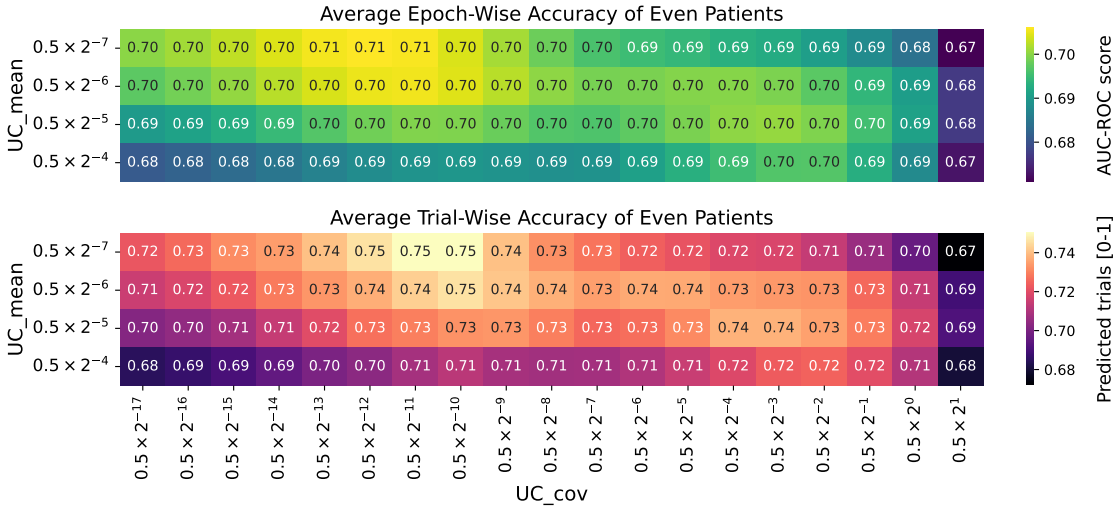


Figure 4.3: Heatmaps of epoch-wise (top) and trial-wise (bottom) accuracy using the data of all even patients. The selected optimal UC-values are $UC_\mu = 0.5 \cdot 2^{-7}$ and $UC_\mu = 0.5 \cdot 2^{-11}$. This UC-pair is used in the *Adaptive CC BT-LDA* on the odd patients.

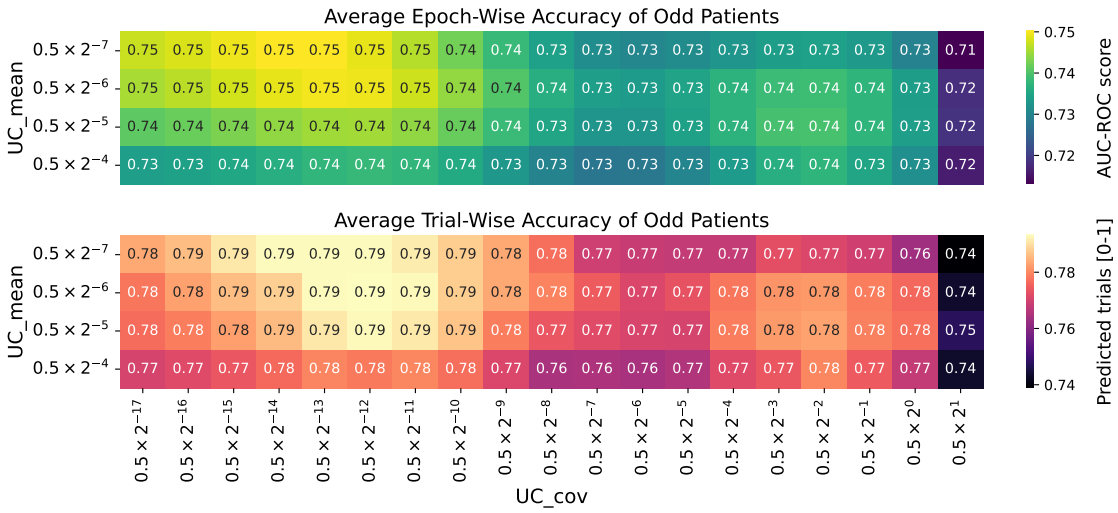


Figure 4.4: Heatmaps of epoch-wise (top) and trial-wise (bottom) accuracy using the data of all odd patients. The selected optimal UC-values are $UC_\mu = 0.5 \cdot 2^{-7}$ and $UC_\mu = 0.5 \cdot 2^{-13}$. This UC-pair is used in the *Adaptive CC BT-LDA* on the even patients.

4.2.3 Optimizing individual UC-pairs

In addition to the two patient groups, a grid search on the UC-pairs has been performed for every individual patient. These results are found in the Appendix. The values are not used in the adaptive classifiers, but rather serve as additional information to assist the discussion of the optimal update coefficient in the next chapter.

4.3 Comparing the Adaptive Classifiers

4.3.1 Window Size for the Adaptive Window BT-LDA

To find the optimal moving window size M the data of the same representative patient as for Section 4.2 was used. The optimal window size was found by combining all sessions of the patient, iterating over different dataset sizes and computing the average AUC-ROC score for each size. For a more robust outcome, multiple trials were taken for every dataset size, after which the average score was returned.

Following the described approach, the average performance for each dataset size was computed and a moving window size of 3600 data points (epochs) was selected, as this value seemed to be located around the global optimum.

Important to mention is that the selected time intervals in the feature processing phase of the pipelines were changed two weeks prior to the thesis deadline. Previously, the time intervals ranged from 0.1 s to 0.5 s in steps of 50 ms. This has now been changed to a range from 0.1 s to 0.8 s in steps of 50 ms. Unfortunately, the restricted time did not allow a recomputation of this approach. So, $M = 3600$ was optimized using the previous time intervals and selected for the Adaptive Window BT-LDA, which uses the new time intervals.

4.3.2 Grand Average Epoch-Wise Performance Over Sessions

The three constructed versions of BT-LDA have been compared against the first baseline Transfer Fixed BT-LDA and second baseline Adaptive CC sLDA. The grand average (across patients) epoch-wise score per session for every classifier against the first baseline is shown in Figure **??**. Likewise, Figure 4.6 shows this performance against the second baseline. Note that the number of patients differs per session, which can be seen in Figure 4.9

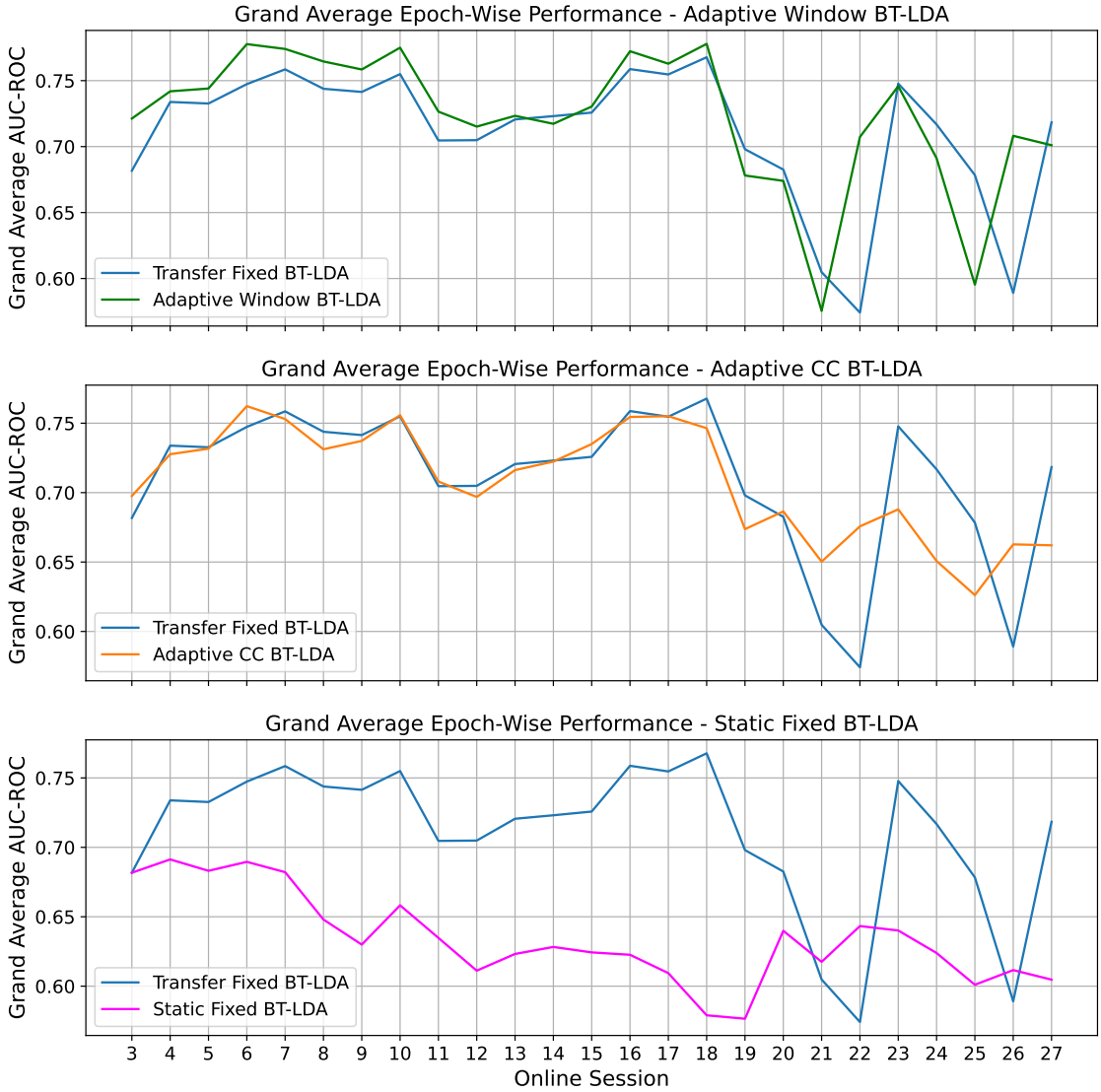


Figure 4.5: The grand average epoch-wise performance of all classifiers compared against the baseline Transfer Fixed BT-LDA. For every session, the scores over patients has been averaged.

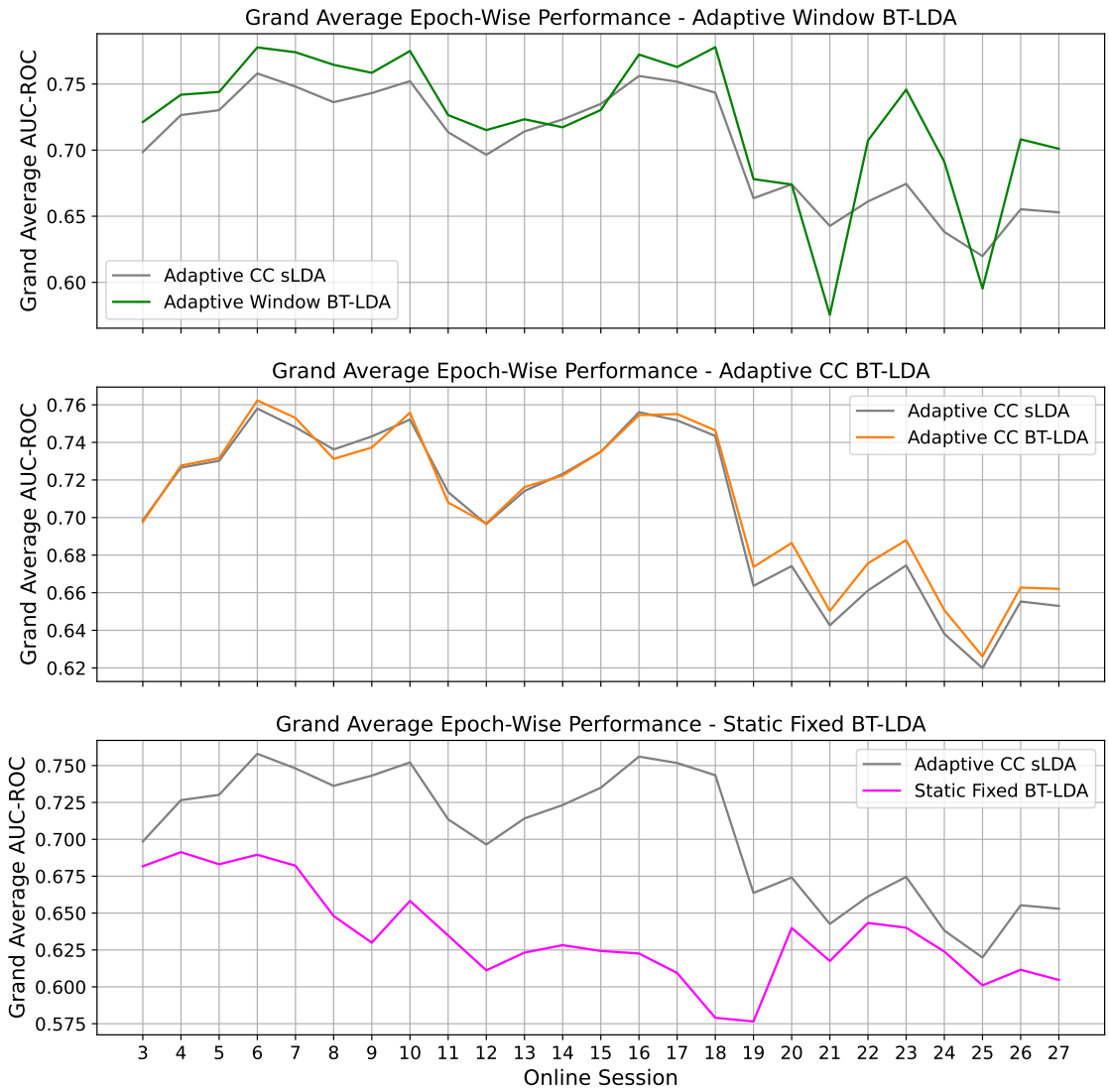


Figure 4.6: The grand average epoch-wise performance of all classifiers compared against the baseline Adaptive CC sLDA. For every session, the scores over patients has been averaged.

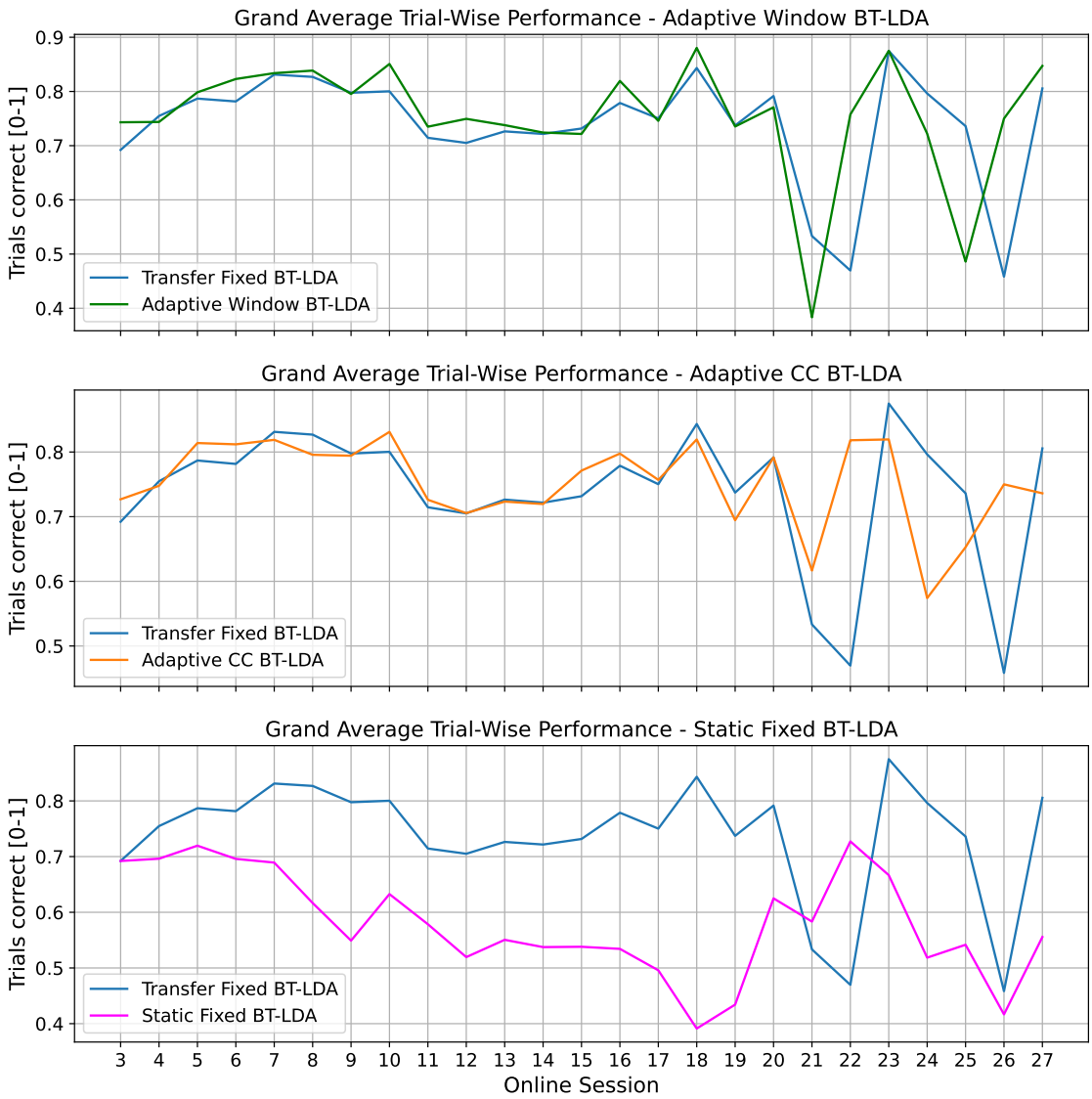


Figure 4.7: The grand average trial-wise performance of all classifiers compared against the baseline Transfer Fixed BT-LDA.

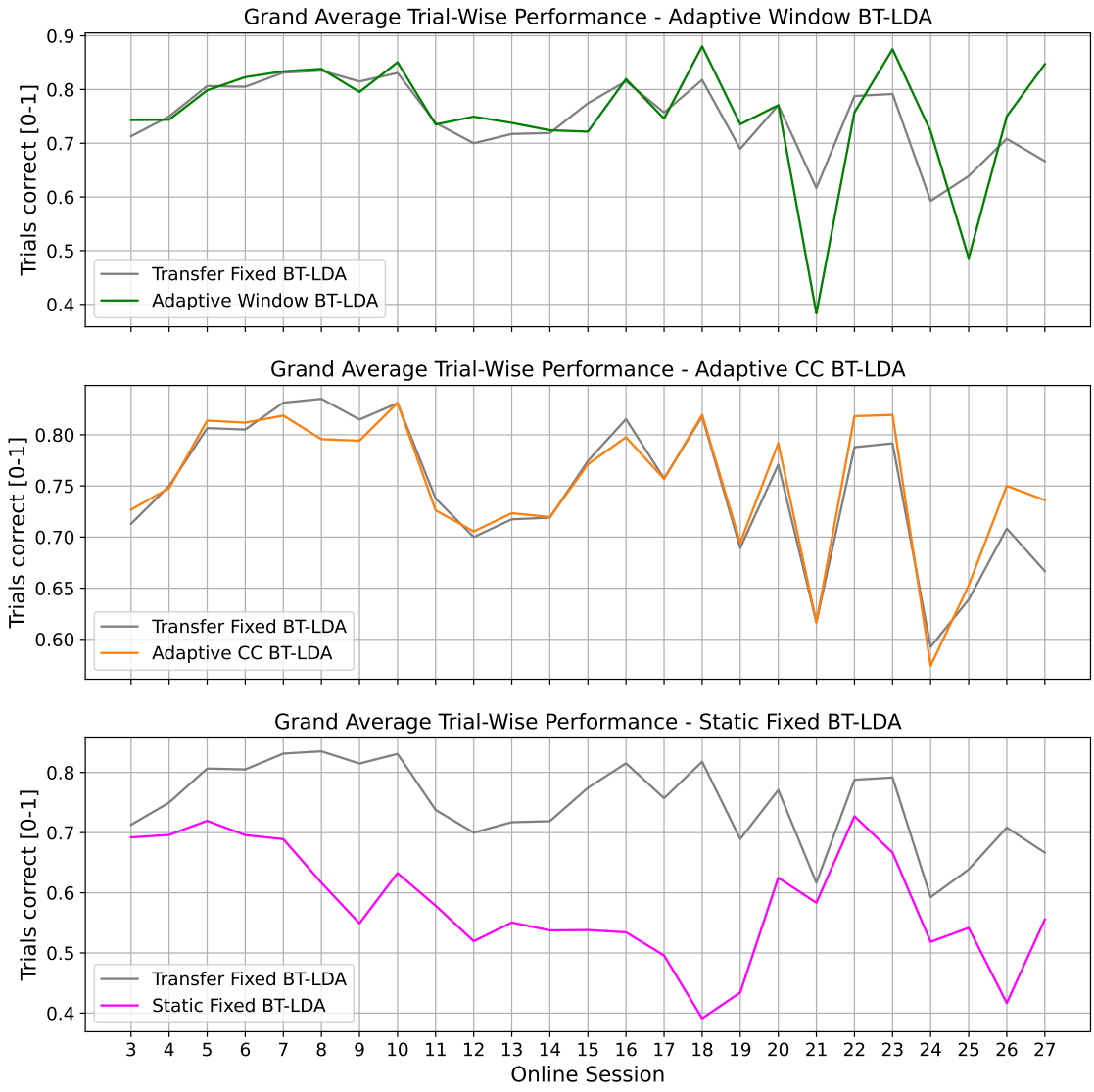


Figure 4.8: The grand average trial-wise performance of all classifiers compared against the baseline Adaptive CC sLDA. For every session, the scores over patients has been averaged.

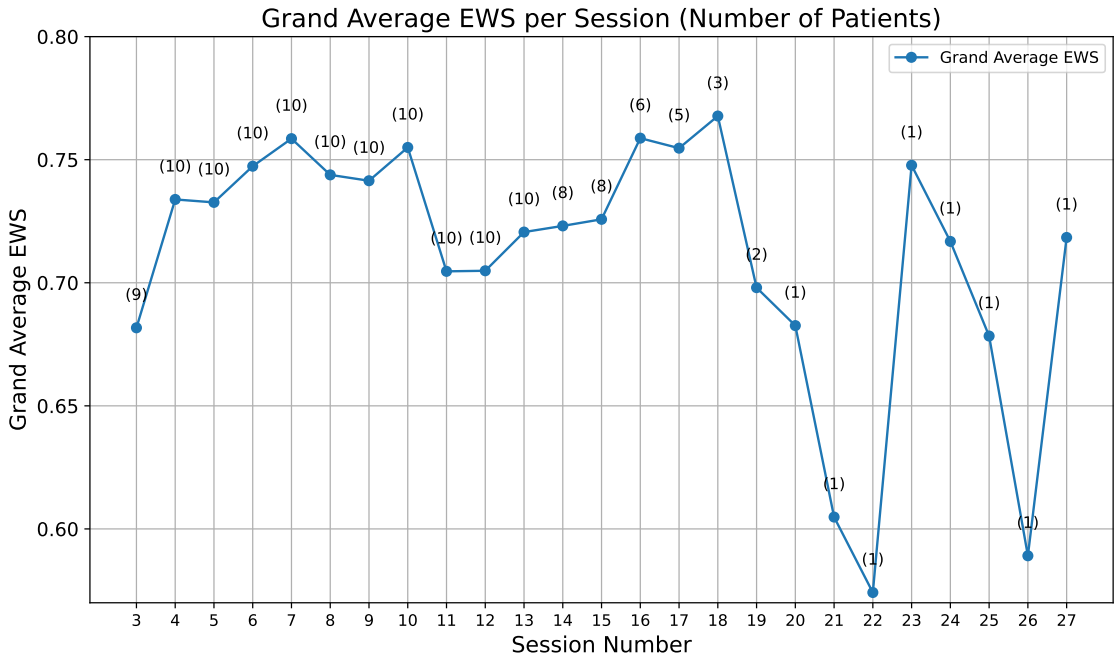


Figure 4.9: The grand average performance of the baseline Transfer Fixed BT-LDA. For each session, the amount of patients is shown, ranging from n = 1 to n = 10.

4.3.3 Statistical Analysis

Three BT-LDA classifiers are compared against two baselines. The best classifier against each baseline is selected, for which a statistical analysis has been conducted. For both baselines, the best classifier appeared to be Adaptive Window BT-LDA. Therefore, 4 statistical tests have been performed: the classifier has been compared against each baseline through a Wilcoxon signed rank test, considering both the epoch-wise and trial-wise performance. The results are shown in Table 4.1.

Table 4.1: Comparison of Adaptive Window BT-LDA against the 1st baseline (Transfer Fixed BT-LDA) and 2nd baseline (Adaptive CC sLDA)

Condition	Mean diff.	Median diff.	Std dev.	Min / Max	Improved sessions	Wilcoxon p-value
EWS - 1st baseline	0.0142	0.0068	0.0446	-0.1171 / 0.1749	90 / 149	6.396×10^{-5}
TWS - 1st baseline	0.0179	0.0000	0.1072	-0.3241 / 0.3958	68 / 149	0.01153
EWS - 2nd baseline	0.0161	0.0140	0.0281	-0.0797 / 0.0882	114 / 149	9.63×10^{-12}
TWS - 2nd baseline	0.0062	0.0069	0.0701	-0.3333 / 0.2500	79 / 149	0.03681

To correct for multiple tests, a Bonferroni correction was used. With the initial significance level $\alpha = 0.05$ and 4 tests (comparing the best classifier with each baseline, for both the epoch-wise and trial-wise score), the adjusted significance level becomes $\alpha_{corrected} = 0.0125$.

Chapter 5

Discussion

This chapter discusses the main findings of this thesis in relation to the research questions and related literature. Possible reasons for the observed results are explored, limitations of this study are considered, and directions for future work are proposed.

5.1 Optimizing the Update Coefficient

Before answering the first research question of this thesis that compares different updating strategies, the hyperparameters of the adaptive classifiers must be determined. Specifically, the optimal update coefficient for the *Adaptive CC BT-LDA* have to be selected. Therefore, this chapter starts with answering the second question of this thesis, which was defined as: What is an optimal update coefficient for the adaptive Block-Toeplitz-LDA?

This update coefficient determines the weight assigned to novel data in the convex combination with the old classifier. In other words, it indicates how strongly old data should be forgotten. As mentioned in Chapter 3, this hyperparameter can be general or subject-specific. For this thesis, a general hyperparameter was chosen. Furthermore, the optimal UC-values for the mean and covariance matrix were found separately through a grid search. The data of the odd patients was used to determine the general UC-pair for the even patients, and vice versa.

5.1.1 The UC-Grid of the Representative Patient

Patient 9 was selected as the representative patient based on the learning curves in Figure (ref), as their learning curve covered a wide range of values for the epoch-wise accuracy and therefore might have represented the remaining patients relatively well. Two 15x15 matrices were constructed for patient 9, one containing the epoch-wise accuracy and one containing the trial-wise accuracy. The performance scores were obtained from a grid search on the original ranges for the UC-pair.

Both matrices present two optima: around ($UC_\mu = 0.5 \cdot 2^{-5},\ UC_\Sigma = 0.5 \cdot 2^{-13}$) and ($UC_\mu = 0.5 \cdot 2^{-5},\ UC_\Sigma = 0.5 \cdot 2^{-1}$). The experimental condition for this patient changed in session 11, which could have caused this observation. To investigate this hypothesis, the dataset of patient 9 was split at session 11, separating the two conditions (see Figure 4.2). However, the separate heatmaps for 6D and HP still showed a grid with two local optima. My supervisor and I assume that this pattern is caused by the fluctuating learning curve of this patient (see Appendix C), where in some sessions the quality of the data was substantially higher than in other sessions.

5.1.2 The UC-Grid of the Grouped Patients

Adjusted UC-ranges

Based on the results of the representative patient, the UC-range for UC_μ and UC_Σ was adjusted separately. The UC_μ showed a single optimum, around which 4 values were selected. The two local optima for UC_Σ covered a wider range and the optimum on the left-hand side seemed interrupted. Therefore, the range for $UC\Sigma$ was increased to 19 values, extending the original range with 4 values on the left.

For further analysis on UC-pairs, individual UC-maps were constructed. These are shown in the Appendix. Based on the individual heatmaps, it seems that the optimal UC-pair varies among subjects. Therefore, selecting a subject-specific UC-pair would probably have increased the chance of obtaining significant improvements over the baseline. This observation aligns with previous work on optimal UC-values [34]. Unfortunately, due to time constraints, this option could not have been explored. However, the insights provided by these individual heatmaps can lay a useful basis for future work on the optimization of these hyperparameters.

Another interpretation of the obtained results is that determining a fixed UC-pair may make the classifier less adaptive, as the optimal forgetting strength would probably change over time, due to non-stationarity.

This insight can be explored by considering adaptive UC-pairs, i.e., UC-values that are not fixed, but rather change over time. For instance, the UC-pair could follow an exponential decay, where weights assigned to novel data reduce over time. Alternatively, dynamic UC-pairs adapt to the quality of novel data. The UC-pair can be updated between sessions, based on the data quality of the previous session, but also adjusting this hyperparameter within sessions is an interesting approach. For instance, the UC-pair can be continuously determined based on the confidence score of a single trial. This score can be obtained through Fisher's LDA score [29] or through a more recent, relatively unexplored approach called unsupervised mean maximization [35].

In this study, the determination of the UC-pair was based on the epoch-wise and trial-wise accuracy (Chapter 3). However, other performance metrics could be used to assess the quality of novel data and assist in choosing the right hyperparameter.

5.2 Optimal Updating Strategy

In the Results Chapter, the Adaptive Window BT-LDA seemed better than the Adaptive CC BT-LDA and was therefore chosen in a statistical analysis where this classifier was compared against the two baselines. Four statistical tests were conducted in total, where in each comparison of Adaptive Window BT-LDA with the baseline, both the epoch-wise and trial-wise accuracy was compared.

With the corrected significance level $a_{corrected} = 0.0125$, Adaptive Window BT-LDA seemed to show significant improvements in the epoch-wise accuracy compared to both baselines. Compared to the Transfer Fixed BT-LDA (1st baseline), the mean difference was 0.0142 with a standard deviation of 0.0446. Compared to the Adaptive CC sLDA (2nd baseline), the mean difference was 0.0161 with a standard deviation of 0.0281.

When comparing the trial-wise performance, Adaptive Window BT-LDA was only significantly better against the Transfer Fixed BT-LDA, with a mean difference of 0.0179 and a standard deviation of 0.1072. The improvements over the second baseline were not

significant.

The results from the offline analysis show that the Adaptive Window BT-LDA has a better epoch-wise (and for one baseline also the trial-wise accuracy) than the baselines on the language rehabilitation dataset used for this study. However, one cannot simply declare one updating strategy better than the other. In fact, multiple factors play a role in the relative performance of each classifier. Different results may be observed on other datasets, pipeline configurations, and evaluation methods.

5.2.1 Evaluation Methods

The classification performance of the proposed classifiers was evaluated through epoch- and trial-wise accuracy. These two metrics appear to correlate, but a number of differences are also shown. The observed inconsistency between the two methods can be explained by their differences.

The epoch-wise accuracy is computed using the AUC-ROC score. This metric is independent of the classifier's bias parameter b, and has a theoretical chance level of 0.5. A value of 0.4, which is 10 AUC points below 0.5, for instance, implies the same classification performance as 0.6, which si 10 points above 0.5.

The trial-wise accuracy is computed as the ratio of correctly predicted trials. Unlike the epoch-wise accuracy, the trial-wise accuracy is a 6-class problem. Furthermore, the performances of each classifier seem to show higher fluctuations in the trial-wise accuracy compared to the epoch-wise one.

5.3 Limitations and Future Directions

5.3.1 Second Baseline: Adaptive CC sLDA

The second baseline of this thesis was the *Adaptive CC sLDA*, which was originally used during the aphasia rehabilitation paradigm. However, in this study, the preprocessing and feature extraction configurations were not completely identical, due to restrictions introduced by BT-LDA. To ensure a fair comparison, I decided to impose these restrictions on the pipeline of this baseline too. Moreover, some preprocessing steps were left out, such as artifact rejection, to compare the most standard case of all classifiers.

Unfortunately, another inconsistency

5.3.2 Offline Analysis

To examine the influence of updating strategies in this thesis, an offline analysis was performed on patient data from a rehabilitation experiment. The original experiment, however, was conducted in an online, closed-loop operation, where there was a continuous interaction between the patient and the BCI system. In this interaction, the user and the system co-adapt, as the user receives feedback from the machine and learns to produce high-quality brain signals, and the machine adapts to their changing brain signals [cite Wolpaw].

Offline analyses do not sufficiently assess how well a new algorithm will perform in an actual online closed-loop setting, since the ongoing interaction between the user and the system cannot be simulated. A different classifier than the one originally used in the experimental setup, may provide different feedback and therefore induce different learning effects on the user. Thus, after finding any successful algorithms from offline analyses, real performance should be assessed in actual online closed-loop evaluations [1].

5.3.3 Intra-session variability in the dataset

The data in this study exhibited variability between sessions, due to several factors. First, patients typically display relatively unstable brain activity patterns ([1], see Appendix C), which may explain fluctuations in their learning curves. Second, as part of the aphasia therapy protocol, the researchers modified the experimental conditions throughout the experiment. Although these factors present a challenge for classification, they compose the nature of the language rehabilitation therapy. Therefore, the classifier must adapt to the variability introduced by these elements.

The pipeline design of the adaptive classifiers in this thesis may insufficiently account for this variability. Perhaps, a more flexible adaptation hyperparameter can be used, which adjusts better to fluctuating brain activity. Moreover, the updating protocol for sessions with a new condition can be further optimized, potentially considering different updating weights or using subject-to-subject transfer learning. In this context, using a dynamic UC-pair within the convex combination strategy remains a promising direction.

5.4 Future Work

5.4.1 Improving the classification in aphasia rehabilitation

In the original language rehabilitation protocol of the dataset, the researchers used a general UC-pair for their adaptive sLDA classifier with $UC_\mu = 0.005$ and $UC_\Sigma = 0.001$. These values were determined from pilot data of healthy subjects.

However, this thesis has shown that the optimal UC-pair substantially differs per subject. Furthermore, a fixed UC-pair does not sufficiently cover the fluctuations in the learning curve of a patient. Lastly, as the recently introduced BT-LDA demonstrated promising improvements over sLDA on smaller datasets [12], an adaptive BT-LDA with a (potentially dynamic) subject-specific UC-pair might surpass the classification performance of sLDA in the language rehabilitation paradigm.

As a consequence of better classification performance, the quality of feedback enhances, which may further improve the language training. It is important to mention, however, that the direct influence of BCI-based feedback on the successful language recovery among patients still has to be investigated [7].

Lastly, this thesis did not aim to optimize the computational efficiency of different update strategies, although in real-time BCIs, this aspect deserves considerable attention. Future work can examine the computational costs of the updating strategies.

5.5 Conclusion

This thesis aimed to find the optimal updating strategy for BT-LDA in the context of a BCI-based language rehabilitation therapy. The primary goals of this thesis include the selection of the optimal update coefficients (UCs) for the Adaptive CC BT-LDA, and a comparative evaluation of two updating stragies: the Adaptive CC BT-LDA and Adaptive Window BT-LDA. These were compared against two baselines.

From the findings, the optimal UC-pair showed to be ideally subject-specific. Furthermore, dynamic UC-values might yield better adaptability to individual learning.

When comparing the update strategies, Adaptive Window BT-LDA generally outperformed the other methods, particularly in epoch-wise accuracy. However, trial-wise accuracy showed more variability and did not demonstrate significant improvements in all

comparisons. This difference highlights the importance of selecting appropriate evaluation metrics when assessing classification performance.

Despite these results, several limitations must be acknowledged. Most importantly, the analysis was performed offline, which does not fully capture the co-adaptive nature of real-time BCI use. Moreover, variability within sessions in the dataset, due to both experimental design and patient-specific factors, presented a challenge that the fixed update strategies could not always address.

Future work should focus on the implementation and evaluation of adaptive classifiers in online, closed-loop settings to better understand their real-world performance. Dynamic update mechanisms, particularly those that adjust UC-values based on data quality or confidence measures, appear to be a promising avenue. Additionally, optimizing computational efficiency remains a crucial step for real-time deployment.

In conclusion, this thesis supports the potential of adaptive update strategies—especially when optimized at the subject level or dynamically adjusted—to improve classification performance in rehabilitation-focused BCI applications. These improvements may contribute not only to more effective BCI systems, but also to enhanced feedback and, ultimately, better rehabilitation outcomes.

Chapter 6

Source Code

The repository with the code for this thesis can be found in `https://github.com/sozraheem/thesis_2025`.

Bibliography

[1] J. R. Wolpaw, N. Birbaumerc, D. J. McFarland, G. Pfurtscheller, and T. M. Vaughan, "Brain–computer interfaces for communication and control," *Clinical Neurophysiology*, vol. 113, no. 1, pp. 767–791, 2002.

[2] L. F. Nicolas-Alonso and J. Gomez-Gil, "Brain Computer Interfaces, a Review," *Sensors*, vol. 12, no. 2, pp. 1211–1279, Feb. 2012, number: 2 Publisher: Molecular Diversity Preservation International. [Online]. Available: https://www.mdpi.com/1424-8220/12/2/1211

[3] M. Khalili-Ardali, S. Wu, A. Tonin, N. Birbaumer, and U. Chaudhary, "Neurophysiological aspects of the completely locked-in syndrome in patients with advanced amyotrophic lateral sclerosis," *Clinical Neurophysiology*, vol. 132, no. 7, pp. 1064–1076, May 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1388245721000341

[4] N. Padfield, J. Zabalza, H. Zhao, V. Masero, and J. Ren, "Eeg-based brain-computer interfaces using motor-imagery: Techniques and challenges," *Sensors*, vol. 19, no. 6, p. 1423, 2019.

[5] W. Jin, X. Zhu, L. Qian, C. Wu, F. Yang, D. Zhan, Z. Kang, K. Luo, D. Meng, and G. Xu, "Electroencephalogram-based adaptive closed-loop brain-computer interface in neurorehabilitation: a review," *Frontiers in Computational Neuroscience*, vol. 18, p. 1431815, 2024.

[6] J. J. Shih, D. J. Krusienski, and J. R. Wolpaw, "Brain-computer interfaces in medicine," in *Mayo clinic proceedings*, vol. 87, no. 3. Elsevier, 2012, pp. 268–279.

[7] M. Musso, D. Hübner, S. Schwarzkopf, M. Bernodusson, P. LeVan, C. Weiller, and M. Tangermann, "Aphasia recovery by language training using a brain–computer interface: A proof-of-concept study," *Brain Communications*, vol. 4, p. fcac008, 2022.

[8] M. F. Mridha, S. C. Das, M. M. Kabir, A. A. Lima, M. R. Islam, and Y. Watanobe, "Brain-computer interface: Advancement and challenges," *Sensors*, vol. 21, no. 17, p. 5746, 2021.

[9] F. Lotte, L. Bougrain, A. Cichocki, M. Clerc, M. Congedo, A. Rakotomamonjy, and F. Yger, "A review of classification algorithms for eeg-based brain–computer interfaces: A 10 year update," *Journal of Neural Engineering*, vol. 15, p. 031005, 2018.

[10] F. Lotte, "Signal processing approaches to minimize or suppress calibration time in oscillatory activity-based brain–computer interfaces," *Proceedings of the IEEE*, vol. 103, pp. 871–890, 2015.

[11] B. Blankertz, S. Lemm, M. Treder, S. Haufe, and K.-R. Müller, "Single-trial analysis and classification of erp components—a tutorial," *NeuroImage*, vol. 56, pp. 814–825, 2011.

[12] J. Sosulski and M. Tangermann, "Introducing block-toeplitz covariance matrices to remaster linear discriminant analysis for event-related potential brain–computer interfaces," *Journal of Neural Engineering*, vol. 19, p. 066001, 2022.

[13] J. Mladenović, J. Mattout, and F. Lotte, "A generic framework for adaptive eeg-based bci training and operation," in *Brain–Computer Interfaces Handbook*. CRC Press, 2018, pp. 595–612.

[14] P. Shenoy, M. Krauledat, B. Blankertz, R. P. Rao, and K.-R. Müller, "Towards adaptive classification for bci," *Journal of neural engineering*, vol. 3, no. 1, p. R13, 2006.

[15] B. Blankertz, M. Kawanabe, R. Tomioka, F. U. Hohlefeld, V. Nikulin, and K.-R. Müller, "Invariant common spatial patterns: alleviating nonstationarities in brain-computer interfacing," in *Neural Information Processing Systems (NIPS)*, 2008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2007/file/6aab1270668d8cac7cef2566a1c5f569-Paper.pdf

[16] M. Tangermann, K.-R. Müller, A. Aertsen, N. Birbaumer, C. Braun, C. Brunner, R. Leeb, C. Mehring, K. J. Miller, G. R. Müller-Putz *et al.*, "Review of the bci competition iv," *Frontiers in neuroscience*, vol. 6, p. 55, 2012.

[17] J. d. R. Millán, R. Rupp, G. R. Müller-Putz, R. Murray-Smith, C. Giugliemma, M. Tangermann, C. Vidaurre, F. Cincotti, A. Kübler, R. Leeb *et al.*, "Combining brain–computer interfaces and assistive technologies: state-of-the-art and challenges," *Frontiers in neuroscience*, vol. 4, p. 161, 2010.

[18] C. Vidaurre and B. Blankertz, "Towards a cure for bci illiteracy," *Brain Topography*, vol. 23, pp. 194–198, 2009.

[19] X. Gu, Z. Cao, A. Jolfaei, P. Xu, D. Wu, T.-P. Jung, and C.-T. Lin, "Eeg-based brain-computer interfaces (bcis): A survey of recent studies on signal sensing technologies and computational intelligence approaches and their applications," *IEEE/ACM transactions on computational biology and bioinformatics*, vol. 18, no. 5, pp. 1645–1666, 2021.

[20] S. Sur and V. K. Sinha, "Event-related potential: An overview," *Industrial psychiatry journal*, vol. 18, no. 1, pp. 70–73, 2009.

[21] D. J. McFarland and J. R. Wolpaw, "Eeg-based brain–computer interfaces," *current opinion in Biomedical Engineering*, vol. 4, pp. 194–200, 2017.

[22] B. Maiseli, A. T. Abdalla, L. V. Massawe, M. Mbise, K. Mkocha, N. A. Nassor, M. Ismail, J. Michael, and S. Kimambo, "Brain–computer interface: trend, challenges, and threats," *Brain informatics*, vol. 10, no. 1, p. 20, 2023.

[23] K. Värbu, N. Muhammad, and Y. Muhammad, "Past, present, and future of eeg-based bci applications," *Sensors*, vol. 22, no. 9, p. 3331, 2022.

[24] A. Gramfort, M. Luessi, E. Larson, D. A. Engemann, D. Strohmeier, C. Brodbeck, R. Goj, M. Jas, T. Brooks, L. Parkkonen *et al.*, "Meg and eeg data analysis with mne-python," *Frontiers in Neuroinformatics*, vol. 7, p. 267, 2013.

[25] J. N. Mak and J. R. Wolpaw, "Clinical applications of brain-computer interfaces: current state and future prospects," *IEEE reviews in biomedical engineering*, vol. 2, pp. 187–199, 2009.

[26] D. J. Krusienski, E. W. Sellers, D. J. McFarland, T. M. Vaughan, and J. R. Wolpaw, "Toward enhanced p300 speller performance," *Journal of neuroscience methods*, vol. 167, no. 1, pp. 15–21, 2008.

[27] J. Höhne, M. Schreuder, B. Blankertz, and M. Tangermann, "Two-dimensional auditory p300 speller with predictive text system," in *2010 Annual international conference of the IEEE engineering in medicine and biology*. IEEE, 2010, pp. 4185–4188.

[28] A. Tharwat, T. Gaber, A. Ibrahim, and A. E. Hassanien, "Linear discriminant analysis: A detailed tutorial," *AI communications*, vol. 30, no. 2, pp. 169–190, 2017.

[29] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*. Springer, 2006, vol. 4, no. 4.

[30] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of multivariate analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[31] H. Xiao and W. B. Wu, "Covariance matrix estimation for stationary time series," 2012.

[32] C. Vidaurre, M. Kawanabe, P. von Bünau, B. Blankertz, and K.-R. Müller, "Toward unsupervised adaptation of lda for brain–computer interfaces," *IEEE Transactions on Biomedical Engineering*, vol. 58, no. 3, pp. 587–597, 2010.

[33] A. Gramfort, M. Luessi, E. Larson, D. A. Engemann, D. Strohmeier, C. Brodbeck, R. Goj, M. Jas, T. Brooks, L. Parkkonen, and M. S. Hämäläinen, "MEG and EEG data analysis with MNE-Python," *Frontiers in Neuroscience*, vol. 7, no. 267, pp. 1–13, 2013.

[34] S. Wu, K. Bhadra, A.-L. Giraud, and S. Marchesotti, "Adaptive lda classifier enhances real-time control of an eeg brain–computer interface for decoding imagined syllables," *Brain Sciences*, vol. 14, no. 3, p. 196, 2024.

[35] J. Sosulski and M. Tangermann, "Umm: Unsupervised mean-difference maximization," *arXiv preprint arXiv:2306.11830*, 2023.

Appendix A

Dataset Information

A.1 Experimental Conditions per Patient

Patient	**Session Details**
1	Sessions 3–17: 6D 350
2	Sessions 3–10: 6D 350 Session 11: 6D 350 (**B1**); 6D 500 (**B2 + B3 (1–5)**) Session 12: 6D 500 Session 13: 6D 350
3	Sessions 3–10: 6D 250 Sessions 11–16: HP 250
4	Sessions 3–11: 6D 350 Session 12: HP 350 (**B1**); HP 500 (**B2 + B3 (1–5)**) Session 13: HP 450 (**B1 (1–2)**); HP 400 (**B1 (3)**); HP 350 (**B1 (4–6)**); HP 350 (**B2 + B3**) Sessions 14–19: HP 350
5	Sessions 3–7: 6D 250 Sessions 8–10: HP 350 Sessions 11–13: HP 250
6	Sessions 3–8: 6D 250 Sessions 9–15: HP 250
7	Sessions 3–27: 6D 350
8	Sessions 4–10: 6D 250 Sessions 11–18: HP 250
9	Sessions 3–10: 6D 350 Sessions 11–12: HP 350 Session 13: HP 350 (**B1 (1–4)**); HP 500 (**B1 (5–6)**); HP 500 (**B2**); HP 750 (**B3 (1–2)**) Session 14: HP 1000 Session 15: HP 650 (**B1 (1–6)**); HP 1000 (**B2 (1–6)**); HP 1000 (**B3 (1)**) Sessions 16–17: HP 1000
10	Sessions 3–15: 6D 350

Table A.1: Overview of the experimental conditions used within every session (explained in Section 3.1.5). B**x** (**y**–**z**) means Block **x**, runs **y** to **z**.

Appendix B

Hyperparameters

B.1 Optimized Time Intervals for BT-LDA

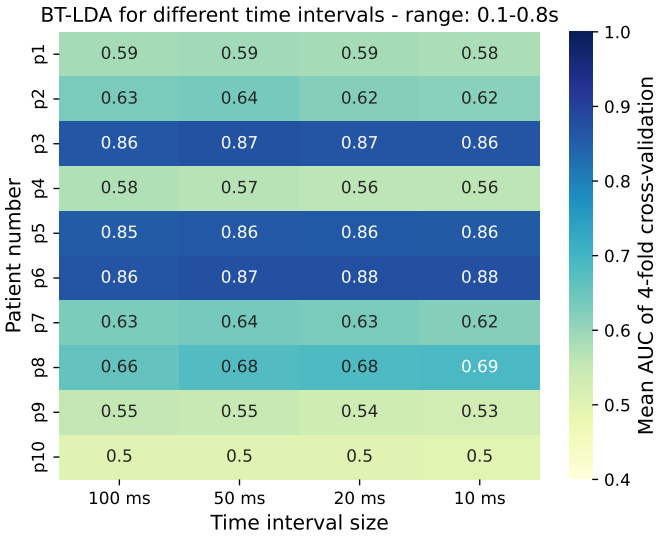
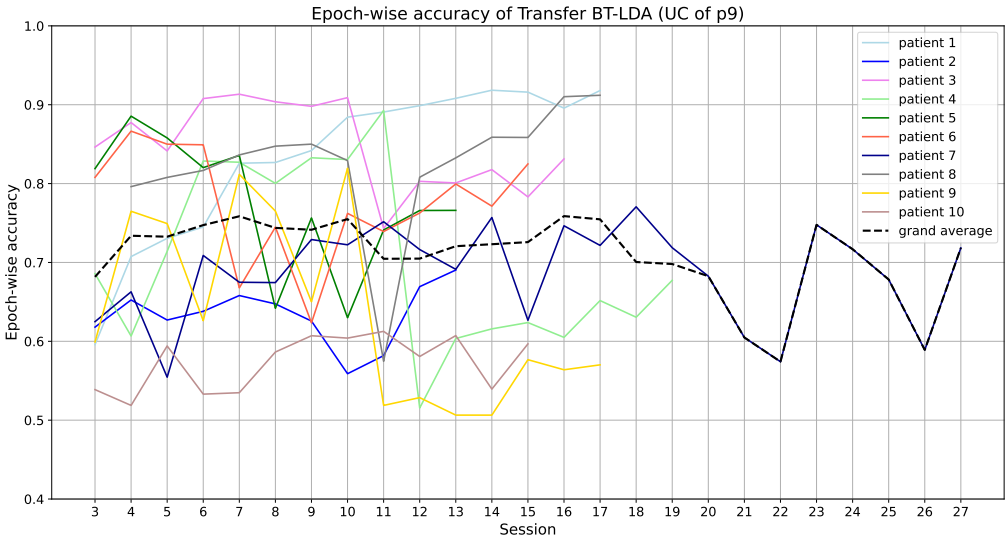


Figure B.1: The average AUC-score of BT-LDA for different time interval sizes. All four time interval configurations range from 0.1 s to 0.8 s. The dataset consists of sessions 1 and 2, selecting only runs with the "6D 350" condition. These results align with the implication that BT-LDA is robust against increasing features in the time domain [12]

Appendix C

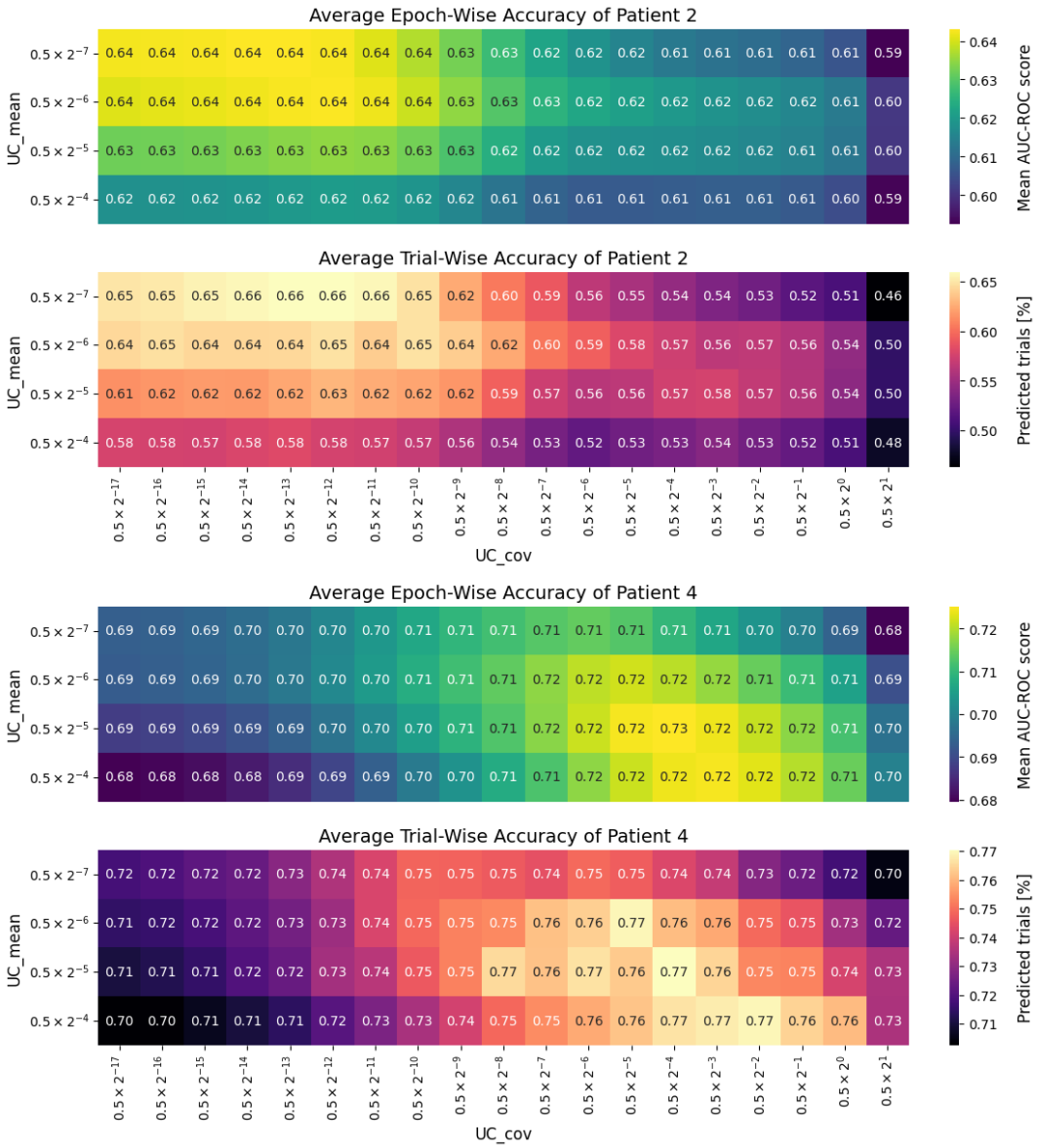
Representative Patient

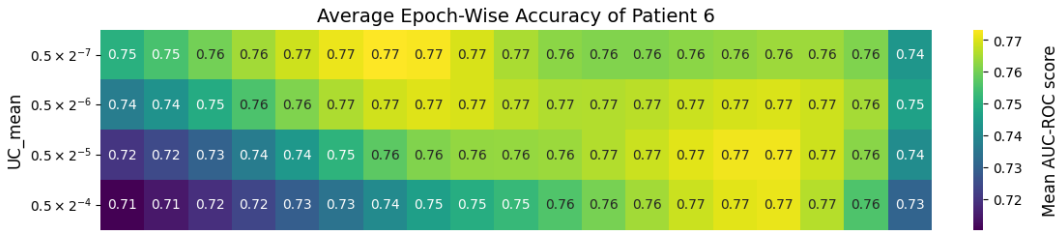


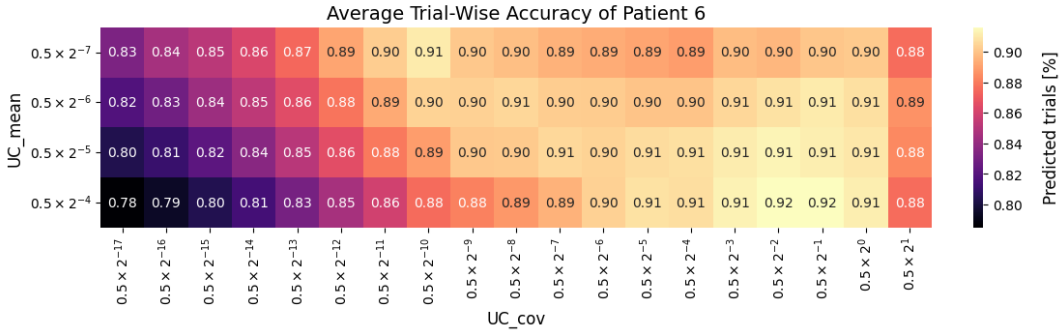
Appendix D

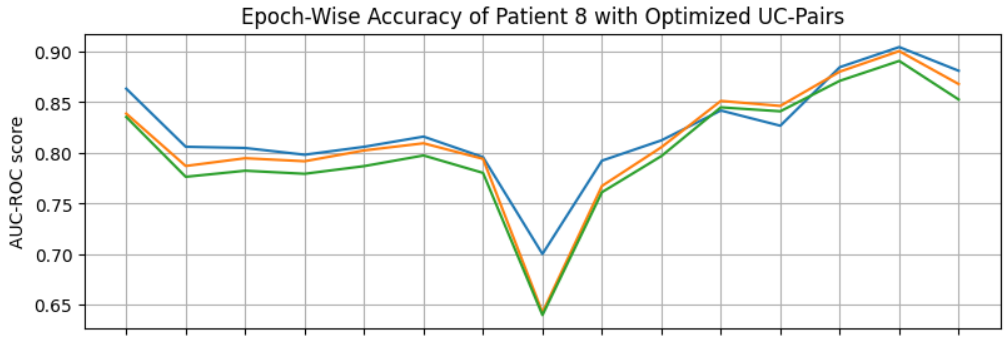
Additional Results

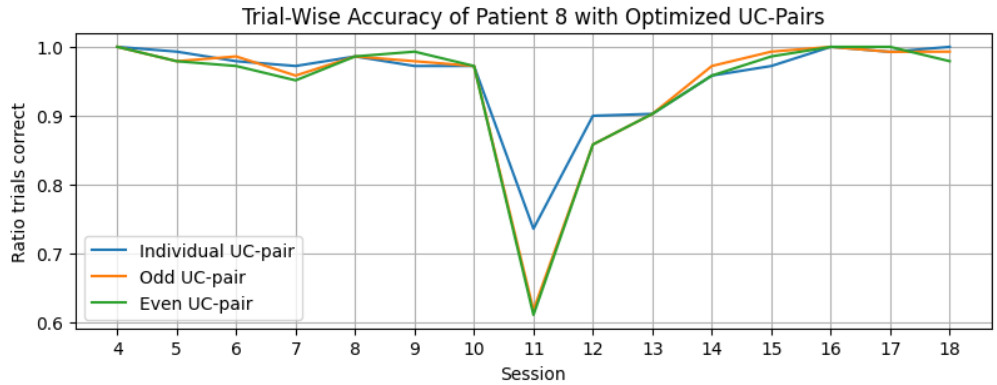
D.1 UC-Pair Grid Search for Every Patient




Average Epoch-Wise Accuracy of Patient 6
UC_mean
0.5 × 2^−7 0.75 0.75 0.76 0.76 0.77 0.77 0.77 0.77 0.77 0.77 0.76 0.76 0.76 0.76 0.76 0.76 0.76 0.76 0.74
0.5 × 2^−6 0.74 0.74 0.75 0.76 0.76 0.77 0.77 0.77 0.77 0.77 0.77 0.77 0.77 0.77 0.77 0.77 0.77 0.76 0.75
0.5 × 2^−5 0.72 0.72 0.73 0.74 0.74 0.75 0.76 0.76 0.76 0.76 0.76 0.77 0.77 0.77 0.77 0.77 0.77 0.76 0.74
0.5 × 2^−4 0.71 0.71 0.72 0.72 0.73 0.73 0.74 0.75 0.75 0.75 0.76 0.76 0.76 0.77 0.77 0.77 0.77 0.76 0.73
0.77 0.76 0.75 0.74 0.73 0.72
Mean AUC-ROC score


Average Trial-Wise Accuracy of Patient 6
UC_mean
0.5 × 2^−7 0.83 0.84 0.85 0.86 0.87 0.89 0.90 0.91 0.90 0.90 0.89 0.89 0.89 0.89 0.90 0.90 0.90 0.90 0.88
0.5 × 2^−6 0.82 0.83 0.84 0.85 0.86 0.88 0.89 0.90 0.90 0.91 0.90 0.90 0.90 0.90 0.91 0.91 0.91 0.91 0.89
0.5 × 2^−5 0.80 0.81 0.82 0.84 0.85 0.86 0.88 0.89 0.90 0.90 0.91 0.90 0.91 0.91 0.91 0.91 0.91 0.91 0.88
0.5 × 2^−4 0.78 0.79 0.80 0.81 0.83 0.85 0.86 0.88 0.88 0.89 0.89 0.90 0.91 0.91 0.91 0.92 0.92 0.91 0.88
0.5 × 2^−17 0.5 × 2^−16 0.5 × 2^−15 0.5 × 2^−14 0.5 × 2^−13 0.5 × 2^−12 0.5 × 2^−11 0.5 × 2^−10 0.5 × 2^−9 0.5 × 2^−8 0.5 × 2^−7 0.5 × 2^−6 0.5 × 2^−5 0.5 × 2^−4 0.5 × 2^−3 0.5 × 2^−2 0.5 × 2^−1 0.5 × 2^0 0.5 × 2^1
UC_cov
0.90 0.88 0.86 0.84 0.82 0.80
Predicted trials [%]


Epoch-Wise Accuracy of Patient 8 with Optimized UC-Pairs
AUC-ROC score
0.90 0.85 0.80 0.75 0.70 0.65


Trial-Wise Accuracy of Patient 8 with Optimized UC-Pairs
Ratio trials correct
1.0 0.9 0.8 0.7 0.6
Individual UC-pair
Odd UC-pair
Even UC-pair
4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
Session